# Two Pointer Technique

Interview Prep Revision Notes

Reverse Array to Trapping Rain Water — Classic Two-Pointer Patterns

## Contents

# Q1 Reverse Array

## INPUT

```
Integer array arr
```

## OUTPUT

Same array reversed.

Matlab: Array ke first element ko last se, second ko second-last se swap karna hai.

## EXAMPLE

Input: [1, 2, 3, 4, 5]

Output: [5, 4, 3, 2, 1]

## ANOTHER EXAMPLE

Input: [10, 20, 30, 40]

Output: [40, 30, 20, 10]

## PATTERN

Two Pointer

Array ke dono ends se start karenge:

```
left  -> first index
right -> last index
```

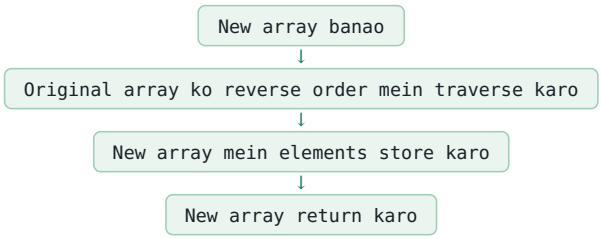
left aur right ke elements ko swap karenge.

Phir:

```
left++
right--
```

Pointers center ki taraf move karenge.

## BRUTE FORCE




Time: O(n)

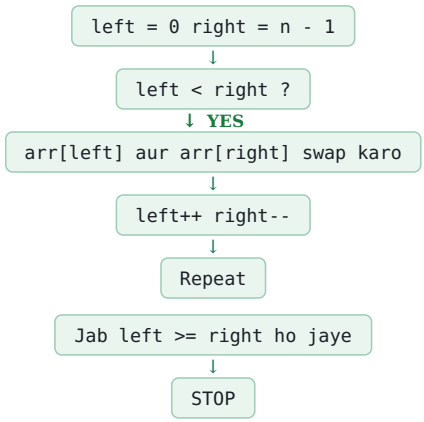
Space: O(n)

Problem:

Extra array use ho raha hai.

## OPTIMIZED APPROACH

Two Pointer + In-place Swap




**IMPORTANT**

Middle element ko separately check nahi karna hai.

Example:

[1, 2, 3, 4, 5]

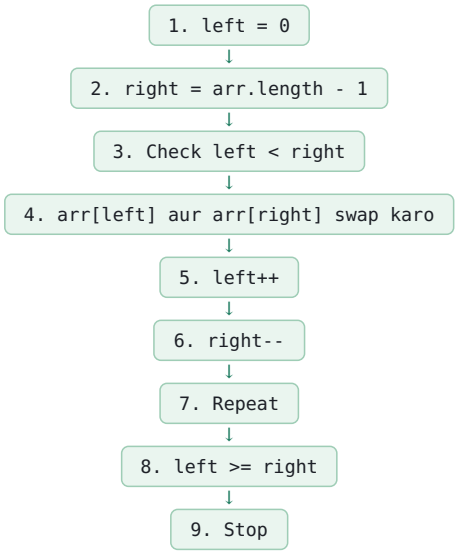
```
   ↑
middle
```

3 automatically correct position par rahega.

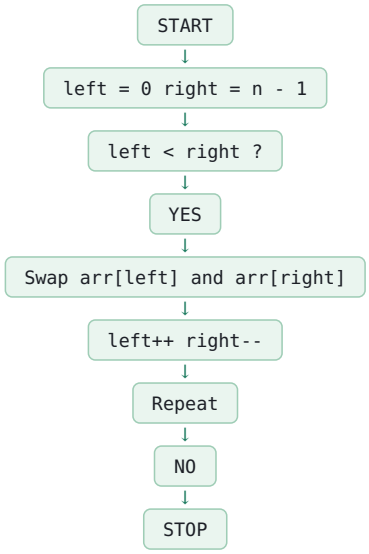
Isliye condition:

```
while (left < right)
```

## STEP-BY-STEP




## FLOWCHART




## DRY RUN

Array:

[1, 2, 3, 4, 5]

Iteration 1:

```
left = 0
right = 4
```

1 ↔ 5

[5, 2, 3, 4, 1]

```
left++
right--

left = 1
right = 3
```

Iteration 2:

2 ↔ 4

[5, 4, 3, 2, 1]

```
left++
right--

left = 2
right = 2
```

Condition:

left < right

2 < 2

FALSE

STOP.

## WRONG

```
int temp = arr[0];
arr[0] = arr[n - 1];
arr[n - 1] = temp;
```

Problem:

Har iteration mein sirf first aur last element hi swap hoga.

## CORRECT

```
int temp = arr[left];
arr[left] = arr[right];
arr[right] = temp;
```

## COMPLETE CODE

```
class Solution {


public void reverseArray(int[] arr) {



    int left = 0;

    int right = arr.length - 1;



    while (left < right) {



        int temp = arr[left];

        arr[left] = arr[right];

        arr[right] = temp;



        left++;

        right--;

    }

}


}
```

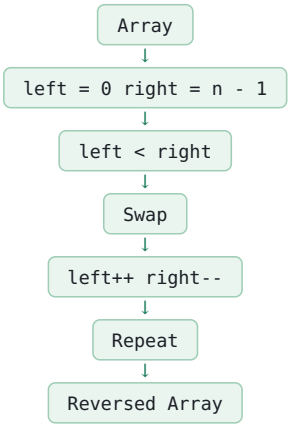
## COMPLEXITY

Time: O(n)

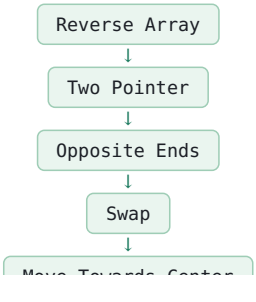
Space: O(1)

Because: Extra array nahi banaya.

## CORE LOGIC




## INTERVIEW PATTERN



Move Towards Center

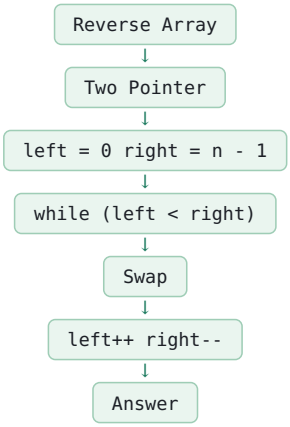
## COMMON MISTAKES

1. 0 se n-1 tak unnecessary loop chalana.

```
Correct:
while (left < right)
```

2. Fixed indices use karna.
Wrong: arr[0], arr[n-1]
Correct: arr[left], arr[right]
3. Middle element ko separately handle karna.
Not required.
4. Extra array banana.
In-place solution mein: O(1) space use karo.

## FINAL REVISION




```
Time  = O(n)
Space = O(1)
```

# Q2 Two Sum II – Sorted Array

## INPUT

```
Sorted integer array numbers
Integer target
```

## OUTPUT

Two indices whose values add up to target.

**IMPORTANT**

Is problem mein indices 1-based hain.

Example:

```
numbers = [2, 7, 11, 15]
target = 9
```

Output:

[1, 2]

Because:

```
2 + 7 = 9
```

## PATTERN

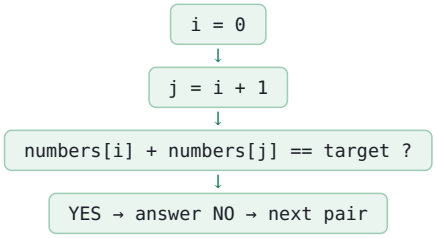
Two Pointer

Because array already sorted hai.

```
left  -> first index
right -> last index
```

## BRUTE FORCE

Har element ko har doosre element ke saath check karo.



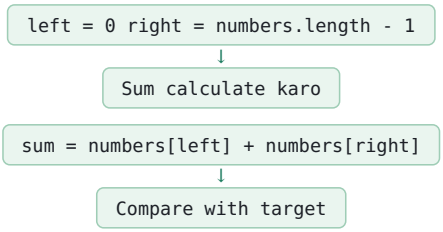

## BRUTE FORCE COMPLEXITY

Time: $O(n^2)$

Space: O(1)

## OPTIMIZED APPROACH

Two Pointers




## CASE 1

sum < target

Matlab sum chhota hai.

Left ko increase karenge:

```
left++
```

WHY?

Array sorted hai.

Left ko right ki taraf move karne par value increase hogi.

Example:

[2, 7, 11, 15] ↑ ↑ left right

```
2 + 15 = 17
```

Agar sum target se chhota hota, to left++ karte.

## CASE 2

sum > target

Matlab sum bada hai.

Right ko decrease karenge:

```
right--
```

WHY?
Array sorted hai.
Right ko left ki taraf move karne par value decrease hogi.

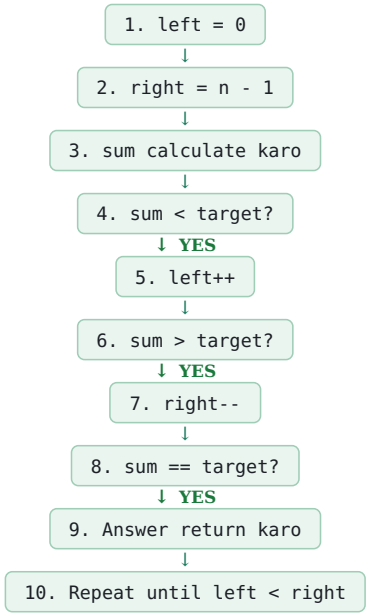
## CASE 3

```
sum == target
```

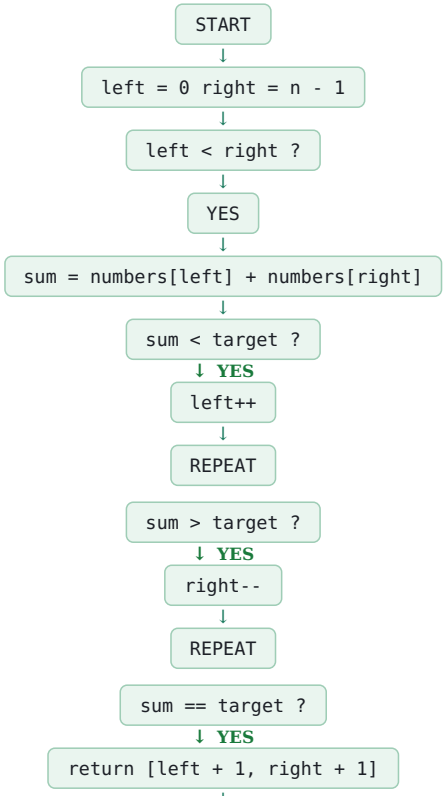
Answer mil gaya.
Return:
left + 1 right + 1

> **IMPORTANT**
> Problem 1-based indices maangti hai.
> Array internally 0-based hai.
> Isliye:
>
> ```
> index 0 → output 1
> index 1 → output 2
> index 2 → output 3
> ```

## STEP-BY-STEP




## FLOWCHART




## DRY RUN

```
numbers = [2, 7, 11, 15]
target = 9
```

Iteration 1:

```
left = 0
right = 3

2 + 15 = 17
```

17 > 9
Therefore:

```
right--
```

Iteration 2:

```
left = 0
right = 2

2 + 11 = 13
```

13 > 9
Therefore:

```
right--
```

Iteration 3:

```
left = 0
right = 1

2 + 7 = 9

9 == 9
```

Answer:
[1, 2]

## COMPLETE FUNCTION

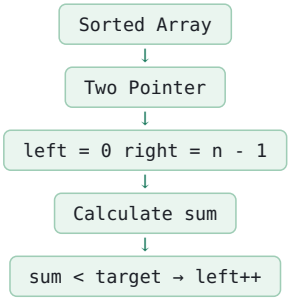
```
public static int[] twoSum(int[] numbers, int target) {


int left = 0;

int right = numbers.length - 1;



while (left < right) {



    int sum = numbers[left] + numbers[right];



    if (sum < target) {



        left++;



    }

    else if (sum > target) {



        right--;



    }

    else {



        return new int[]{left + 1, right + 1};

    }

}
```

```
return new int[]{-1, -1};

}
```

## COMPLEXITY

Time:
O(n)
Because:
left only moves forward and right only moves backward.
Space:
O(1)
No extra data structure used.

## CORE LOGIC




```
sum > target
→ right--

sum == target
→ return answer
```

## IMPORTANT

Sorted array hone ki wajah se Two Pointer possible hai.
Agar array sorted nahi hota, to ye exact approach directly use nahi kar sakte.

## COMMON MISTAKES

1. Wrong:

```
left = 1
```

Correct:

```
left = 0
```

2. Wrong:

```
right = numbers.length
```

Correct:

```
right = numbers.length - 1
```

3. Wrong:

```
sum < target → right--
```

Correct:

```
sum < target → left++
```

4. Wrong:

```
sum > target → left++
```

Correct:

```
sum > target → right--

5. 0-based indices return karna.
```

Wrong:

```
return new int[]{left, right};
```

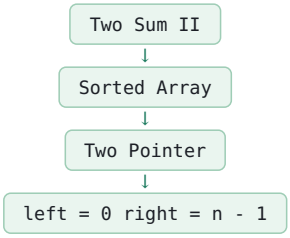
Correct:

```
return new int[]{left + 1, right + 1};
```

6. Array sorted hone ki condition ignore karna.

> Two Pointer ka main advantage sorted order ki wajah se mil raha hai.

## FINAL REVISION




```
sum < target
→ left++

sum > target
→ right--

sum == target
→ answer

Time  = O(n)
Space = O(1)
```

# Q3 Remove Duplicates From Sorted Array

## INPUT

Sorted integer array nums

## OUTPUT

Unique elements ki count k

Array ke first k elements mein saare unique elements hone chahiye.

> **IMPORTANT**
>
> Array ko in-place modify karna hai.
>
> Extra array/list nahi banana hai.

## EXAMPLE

Input:

[1, 1, 2, 2, 3]

Output:

```
k = 3
```

First 3 elements:

[1, 2, 3]

## ANOTHER EXAMPLE

Input:

[0, 0, 1, 1, 1, 2, 2, 3]

Output:

```
k = 4
```

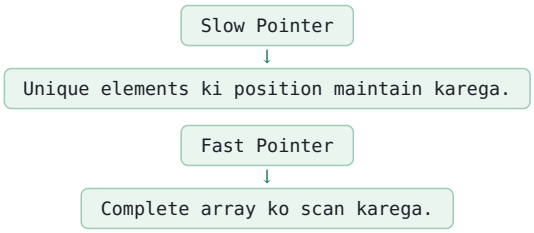
First 4 elements:

[0, 1, 2, 3]

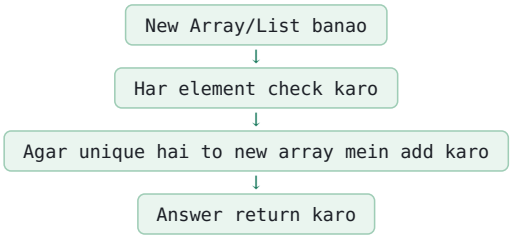
> **KEY OBSERVATION**
>
> Array SORTED hai.
>
> Isliye duplicates hamesha adjacent/side-by-side honge.
>
> Example:
>
> [1, 1, 2, 2, 3, 3] ↑ ↑ duplicate

## PATTERN

Two Pointer




## BRUTE FORCE




Problem:

Extra Space lagegi.

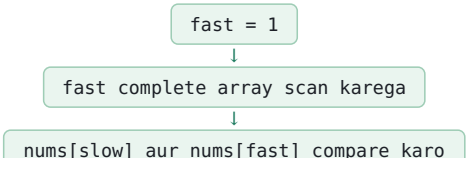
Space:

O(n)

## OPTIMIZED APPROACH

Slow + Fast Pointer

```
slow = 0
```



## CASE 1

```
nums[slow] == nums[fast]
```

Matlab duplicate hai.
↓
Kuch nahi karna.
↓
Fast automatically next iteration mein move karega.

## CASE 2

```
nums[slow] != nums[fast]
```

Matlab new unique element mila.
↓
slow++
↓
nums[slow] = nums[fast]

Isse unique element array ke front part mein store ho jayega.

**IMPORTANT**

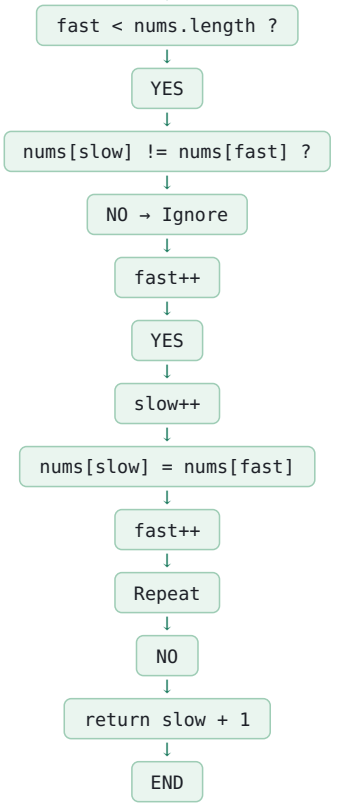
Hum duplicate ko delete nahi kar rahe.
Hum sirf unique elements ko array ke starting portion mein overwrite kar rahe hain.
Example:
[1, 1, 2, 2, 3]
After processing:
[1, 2, 3, 2, 3]
Useful portion:
[1, 2, 3]
Return:
3

## STEP-BY-STEP

1. Empty array check karo.
↓
2. slow = 0
↓
3. fast = 1
↓
4. fast ko array ke end tak chalao.
↓
5. nums[slow] == nums[fast] ?
↓ YES
6. Duplicate → ignore.
↓
7. nums[slow] != nums[fast] ?
↓ YES
8. slow++
↓
9. nums[slow] = nums[fast]
↓
10. Fast next element par jayega.
↓
11. Loop complete.
↓
12. return slow + 1

## FLOWCHART

START
↓
nums.length == 0 ?
↓
YES → return 0
↓ NO
slow = 0
↓
fast = 1
↓




## DRY RUN

nums:

[1, 1, 2, 2, 3]

## START

```
slow = 0
fast = 1
```

Iteration 1:

```
nums[slow] = 1
nums[fast] = 1

1 == 1
```

Duplicate.

```
→ Ignore.
```

Iteration 2:

```
slow = 0
fast = 2

nums[slow] = 1
nums[fast] = 2

1 != 2

→ slow++

slow = 1

→ nums[1] = nums[2]
```

Array:

[1, 2, 2, 2, 3]

Iteration 3:

```
slow = 1
fast = 3

2 == 2
```

Duplicate.

```
→ Ignore.
```

Iteration 4:

```
slow = 1
fast = 4

2 != 3
```

```
→ slow++

slow = 2

→ nums[2] = nums[4]
```

Array:
[1, 2, 3, 2, 3]
Loop complete.
Return:
slow + 1

```
= 2 + 1

= 3
```

Useful portion:
[1, 2, 3]

## COMPLETE FUNCTION

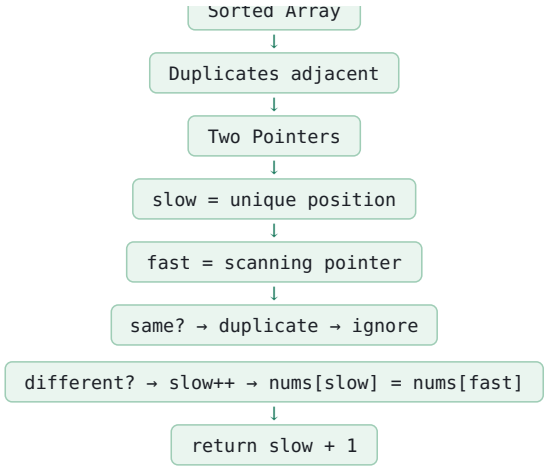
```
public static int removeDuplicates(int[] nums) {


if (nums.length == 0) {

    return 0;

}



int slow = 0;



for (int fast = 1; fast < nums.length; fast++) {



    if (nums[slow] != nums[fast]) {



        slow++;

        nums[slow] = nums[fast];

    }

}



return slow + 1;


}

WHY RETURN slow + 1?

slow index hai,
count nahi.

Example:

slow = 0
→ 1 unique element

slow = 1
→ 2 unique elements

slow = 2
→ 3 unique elements

Therefore:

unique count = slow + 1
```

## COMPLEXITY

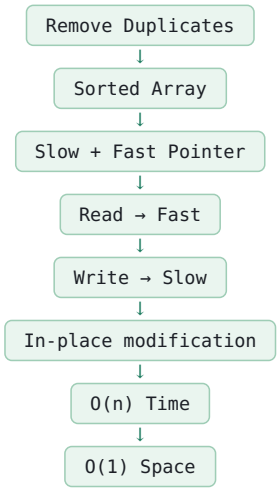
Time:
O(n)
Fast pointer complete array ko sirf ek baar scan karta hai.
Space:
O(1)
Extra array/list nahi banaya.

## CORE LOGIC

Sorted Array




## INTERVIEW PATTERN




### COMMON MISTAKES

1. New Array/List banana.
Question in-place hai.
Avoid extra space.
2. Duplicate milne par slow ko move karna.
Wrong.
Duplicate par:
slow same rahega.
3. Unique element milne par slow ko increment na karna.
Correct:

```
slow++
nums[slow] = nums[fast]

4. return slow karna.
```

Wrong.
Correct:

```
return slow + 1
```

5. Array sorted hone ka advantage use na karna.
Sorted array mein duplicates adjacent hote hain.

### IMPORTANT DIFFERENCE

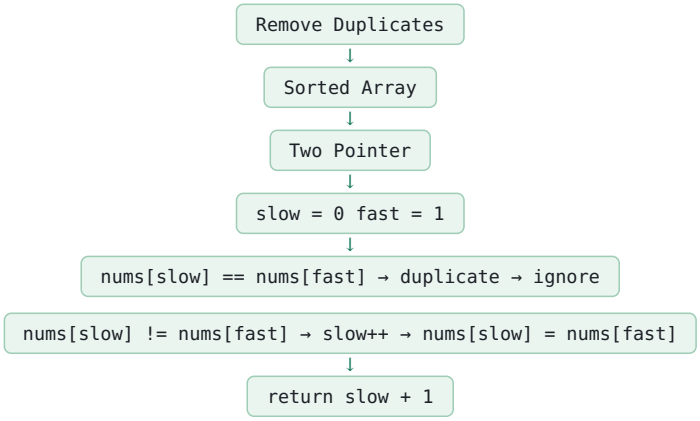
Q1 Reverse Array:
left + right
Opposite directions:

```
left →
← right
```

Q3 Remove Duplicates:
slow + fast
Same direction:

```
slow →
fast →
```

## FINAL REVISION




```
Time  = O(n)
Space = O(1)
```

# Q4 Move Zeroes

## INPUT

```
Integer array arr
```

## OUTPUT

Saare zeroes ko array ke end mein move karna hai.
Non-zero elements ka relative order same rehna chahiye.

### IMPORTANT

Array ko in-place modify karna hai.
Extra array use nahi karna hai.

## EXAMPLE

Input:
[0, 1, 0, 3, 12]
Output:
[1, 3, 12, 0, 0]

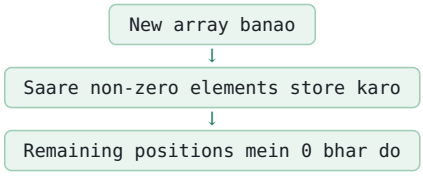
## ANOTHER EXAMPLE

Input:
[0, 0, 1]
Output:
[1, 0, 0]

### IMPORTANT OBSERVATION

Hume non-zero elements ko array ke left side mein arrange karna hai.
Zeroes automatically end mein chale jayenge.

## BRUTE FORCE




Problem:
Extra space lagegi.
Space: O(n)

## OPTIMIZED APPROACH

Two Pointer
Hum first zero ka index `j` find karenge.
j: Next position jahan non-zero element place karna hai.
i: Array ko scan karega.

## STEP 1

First zero find karo.
Example:
[0, 1, 0, 3, 12]

```
j = 0
```

## STEP 2

j ke baad se array scan karo.

```
i = j + 1
```

## STEP 3

Agar:

```
arr[i] == 0
```

Toh:
Kuch nahi karna.
Sirf i next position par jayega.

## STEP 4

Agar:

```
arr[i] != 0
```

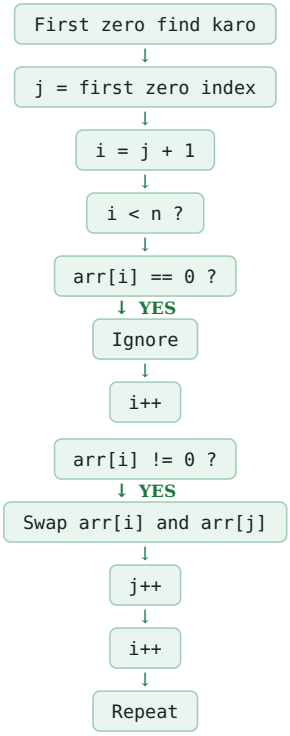
Toh non-zero element mila.

Swap:
arr[i] ↔ arr[j]
Phir:

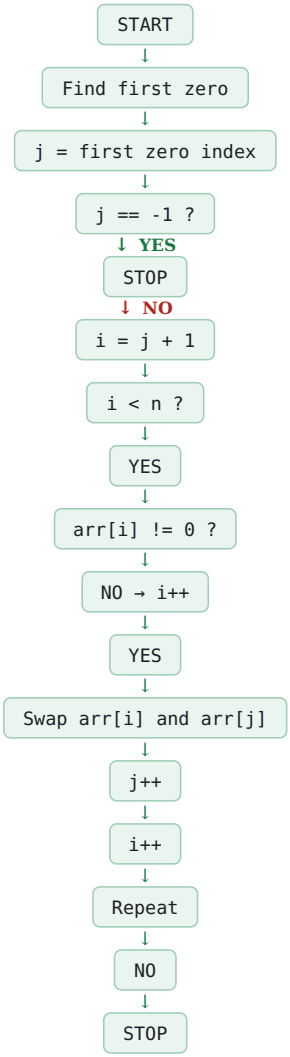
```
j++
```

WHY j++?
Kyuki current zero position fill ho gayi.
Ab next zero position par next non-zero element place karna hai.

## WORKFLOW




## FLOWCHART



## DRY RUN

Input:
[0, 1, 0, 3, 12]
First zero:

```
j = 0

i = 1

arr[i] = 1

1 != 0
```

Swap:
[1, 0, 0, 3, 12]

```
j++
```

Now:

```
j = 1

i = 2

arr[i] = 0
```

Ignore.

```
i = 3

arr[i] = 3

3 != 0
```

Swap:
[1, 3, 0, 0, 12]

```
j++
```

Now:

```
j = 2

i = 4

arr[i] = 12

12 != 0
```

Swap:
[1, 3, 12, 0, 0]

```
j++
```

## FINAL

[1, 3, 12, 0, 0]

## COMPLETE FUNCTION

```
public static void moveZeroes(int[] arr) {

int n = arr.length;


int j = -1;


// First zero find karo

for (int i = 0; i < n; i++) {


    if (arr[i] == 0) {

        j = i;

        break;

    }

}


// Array mein zero nahi hai
```

```
if (j == -1) {

    return;

}


// First zero ke baad scan karo

for (int i = j + 1; i < n; i++) {


    if (arr[i] != 0) {


        int temp = arr[i];

        arr[i] = arr[j];

        arr[j] = temp;


        j++;

    }

}


}
```

## COMPLEXITY

Time:

O(n)

Array ko maximum do baar scan kiya ja raha hai.

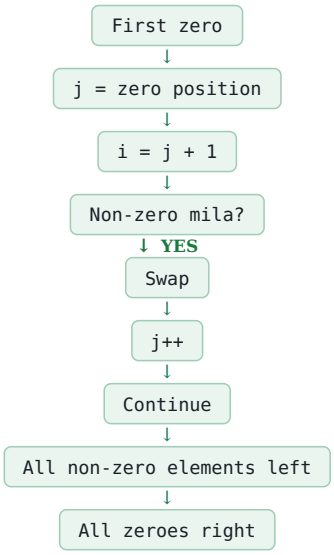
```
O(n) + O(n)
= O(n)
```

Space:

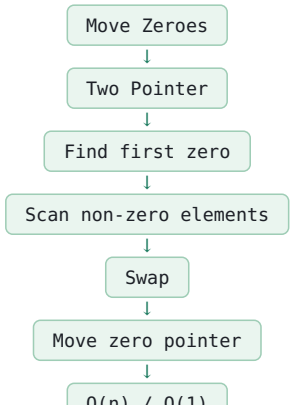
O(1)

Extra array ya data structure use nahi hua.

## CORE LOGIC




## INTERVIEW PATTERN



O(n) / O(1)

**IMPORTANT**

`j` value nahi hai.

`j` index/pointer hai.

Correct:

```
int j = 0;
```

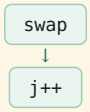
Ya current approach mein:

```
int j = -1;
```

**COMMON MISTAKES**

1. Swap ke baad j++ bhool jaana.

Correct:




2. Zero milne par j++ kar dena.

Wrong.

Zero ko ignore karna hai.

`j` ko tabhi move karna hai jab non-zero element milkar zero position fill kare.

3. Non-zero elements ka order change karna.

Required order:

```
1 → 3 → 12
```

same rehna chahiye.

4. Extra array use karna.

In-place solution mein:

O(1) space

```
5. Zero hi na ho to j = -1 rehna.

Is case mein:

if (j == -1) {
return;
}
```

## STANDARD INTERVIEW VERSION

Is problem ka aur simpler standard Two Pointer solution bhi hai:

```
j = 0

i = 0 se scan karo.
```

Agar:

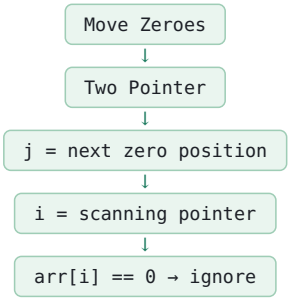
```
arr[i] != 0
```

To:

```
swap(arr[i], arr[j])
j++
```

Ye version interview mein zyada commonly use kiya ja sakta hai.

## FINAL REVISION




```
arr[i] != 0
→ swap(arr[i], arr[j])
→ j++
```

Final:

```
Non-zero elements → left
```

```
Zeroes → right

Time  = O(n)
Space = O(1)
```

# Q5 Squares Of A Sorted Array

## INPUT

Sorted integer array nums

## OUTPUT

```
Har element ka square
sorted order mein return karna hai.
```

## EXAMPLE

Input:
[-4, -1, 0, 3, 10]
Output:
[0, 1, 9, 16, 100]

## ANOTHER EXAMPLE

Input:
[-7, -3, 2, 3, 11]
Output:
[4, 9, 9, 49, 121]

### IMPORTANT OBSERVATION

Input array sorted hai.
Lekin square karne ke baad array sorted nahi rahega.
Example:
[-4, -1, 0, 3, 10]
Squares:
[16, 1, 0, 9, 100]
Isliye simple left-to-right square karna enough nahi hai.

### KEY OBSERVATION

Largest square hamesha array ke left ya right end se milega.
Example:
[-4, -1, 0, 3, 10]
Left:

```
-4² = 16
```

Right:

```
10² = 100
```

Dono ends compare karne hain.

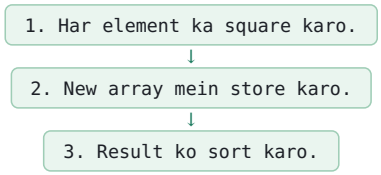
## PATTERN

Two Pointer

```
left  -> first index
right -> last index
```

Result array ke end se fill karenge.

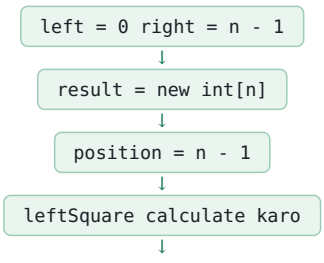
## BRUTE FORCE

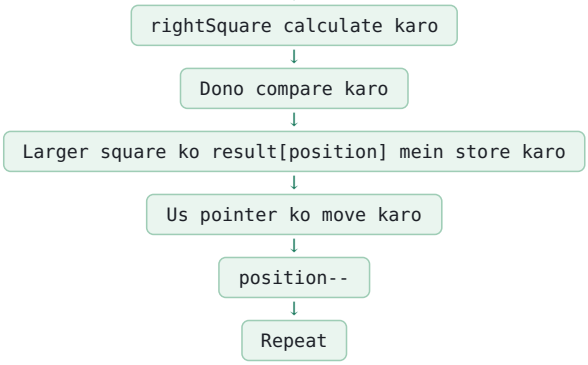



Complexity:
Time: O(n log n)
Space: O(n)

## OPTIMIZED APPROACH

Two Pointer



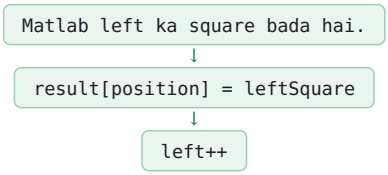



## SQUARE CALCULATION

leftSquare:
nums[left] * nums[left]
rightSquare:
nums[right] * nums[right]

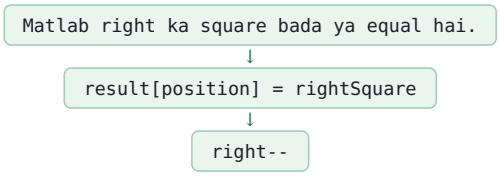
## CASE 1

leftSquare > rightSquare




## CASE 2

```
rightSquare >= leftSquare
```



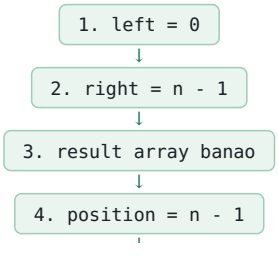

## HAR ITERATION

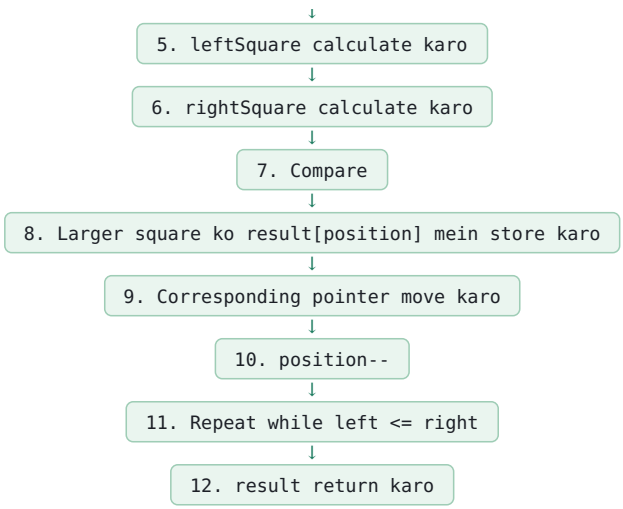
position--
WHY RESULT END SE FILL KARTE HAIN?
Har iteration mein sabse bada remaining square mil raha hai.
Isliye:

```
Largest → last position

Second largest → second-last

Third largest → third-last
```

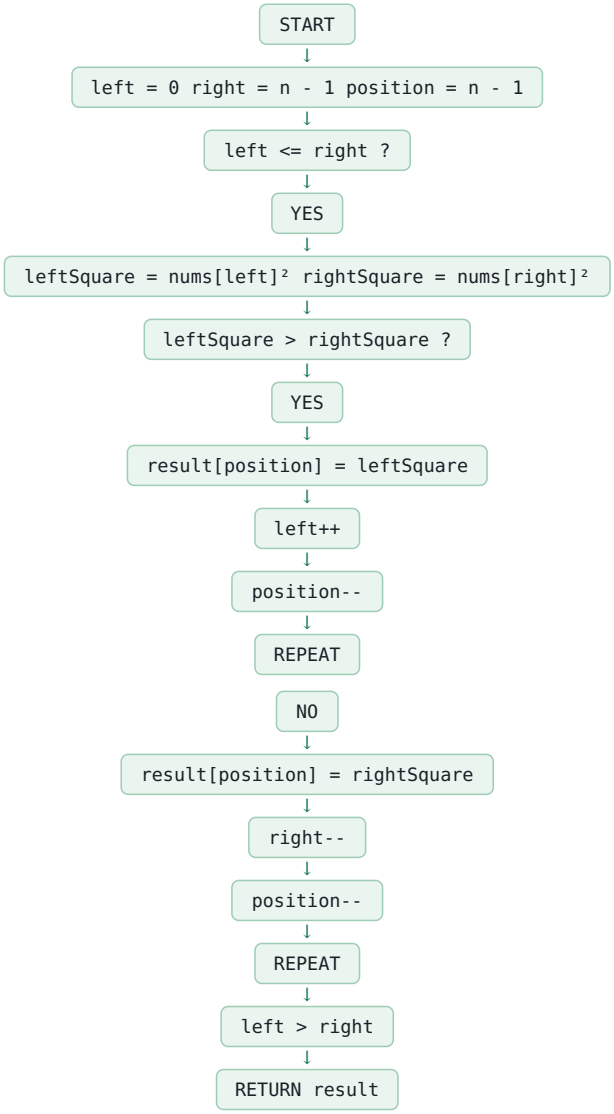
Example:
[-4, -1, 0, 3, 10]
First:
16 vs 100
100 largest hai.
result:
[_, _, _, _, 100]
Then:
16 vs 9
16 largest hai.
result:
[_, _, _, 16, 100]
Eventually:
[0, 1, 9, 16, 100]

## STEP-BY-STEP






## FLOWCHART




## DRY RUN

nums:
[-4, -1, 0, 3, 10]

## START

```
left = 0
right = 4
position = 4
```

Iteration 1:
leftSquare:

```
(-4)² = 16
```

rightSquare:

```
10² = 100
```

100 bigger.

```
result[4] = 100

right--

right = 3

position = 3
```

Iteration 2:
leftSquare:

```
(-4)² = 16
```

rightSquare:

```
3² = 9
```

16 bigger.

```
result[3] = 16

left++

left = 1

position = 2
```

Iteration 3:
leftSquare:

```
(-1)² = 1
```

rightSquare:

```
3² = 9
```

9 bigger.

```
result[2] = 9

right--

right = 2

position = 1
```

Iteration 4:
leftSquare:

```
0² = 0
```

rightSquare:

```
0² = 0
```

Equal.
Else case:

```
result[1] = 0

right--

right = 1

position = 0
```

Iteration 5:
leftSquare:

```
(-1)² = 1
```

rightSquare:

```
(-1)² = 1

result[0] = 1

right--
```

STOP.

> **NOTE**
> Dry run mein pointer movement ko carefully track karna important hai.
> Final sorted result:
> [1, 0, 9, 16, 100]
> For this particular sequence, the above dry-run ordering reveals that when equal squares occur, we must ensure the remaining smaller values are placed correctly.

## STANDARD CORRECT DRY RUN

For:

[-4, -1, 0, 3, 10]

Iteration 1:

```
16 vs 100
→ result[4] = 100
→ right--
```

Iteration 2:

```
16 vs 9
→ result[3] = 16
→ left++
```

Iteration 3:

```
1 vs 9
→ result[2] = 9
→ right--
```

Iteration 4:

```
1 vs 0
→ result[1] = 1
→ left++
```

Iteration 5:

```
0 vs 0
→ result[0] = 0
```

Final:

[0, 1, 9, 16, 100]

## COMPLETE FUNCTION

```
public static int[] sortedSquares(int[] nums) {


int left = 0;

int right = nums.length - 1;



int[] result = new int[nums.length];



int position = nums.length - 1;



while (left <= right) {



    int leftSquare = nums[left] * nums[left];

    int rightSquare = nums[right] * nums[right];



    if (leftSquare > rightSquare) {



        result[position] = leftSquare;

        left++;



    }

    else {



        result[position] = rightSquare;

        right--;

    }



    position--;

}
```

```
return result;

}
```

## COMPLEXITY

Time:

O(n)

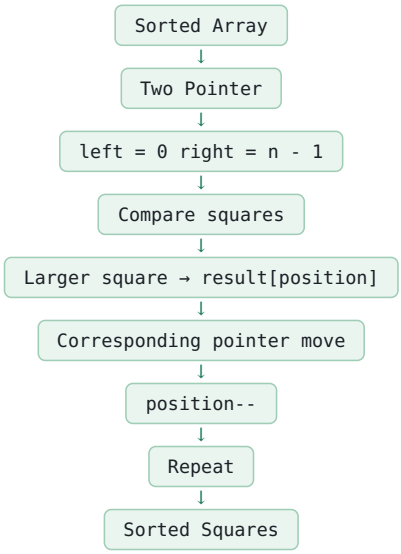
Har element maximum ek baar process hota hai.

Space:

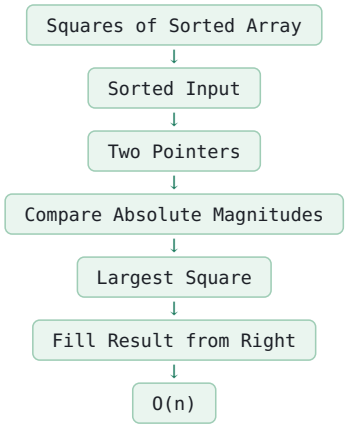
O(n)

Result array use ho raha hai.

## CORE LOGIC




## INTERVIEW PATTERN




## COMMON MISTAKES

1. Input ko directly square karke sorted maan lena.

Wrong:

[-4, -1, 0, 3]

Squares:

[16, 1, 0, 9]

Ye sorted nahi hai.

2. Result ko left se fill karna.

Largest square pehle mil raha hai.

Isliye result ko right se fill karo.

3. Negative values ka square correctly calculate na karna.

Correct:

nums[left] * nums[left]

4. `position--` bhool jaana.

Har iteration ke baad:

position--

5. Wrong loop condition.

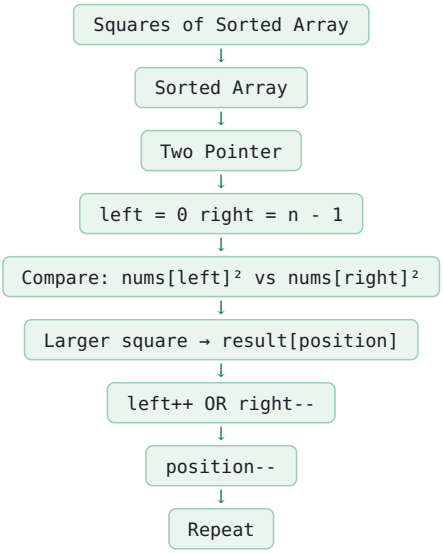
Correct:

```
while (left <= right)
```

```
WHY `<=`?
```

Jab last ek element bachta hai, left aur right same ho sakte hain.
Us element ko bhi process karna hai.

## FINAL REVISION




```
Time  = O(n)
Space = O(n)
```

# Q5 Squares Of A Sorted Array

## INPUT

Sorted integer array nums

## OUTPUT

```
Har element ka square
sorted order mein return karna hai.
```

## EXAMPLE

Input:
[-4, -1, 0, 3, 10]
Output:
[0, 1, 9, 16, 100]

## ANOTHER EXAMPLE

Input:
[-7, -3, 2, 3, 11]
Output:
[4, 9, 9, 49, 121]

### IMPORTANT OBSERVATION

Input array sorted hai.
Lekin square karne ke baad array sorted nahi rahega.
Example:
[-4, -1, 0, 3, 10]
Squares:
[16, 1, 0, 9, 100]
Isliye simple left-to-right square karna enough nahi hai.

### KEY OBSERVATION

Largest square hamesha array ke left ya right end se milega.
Example:
[-4, -1, 0, 3, 10]
Left:

```
-4² = 16
```

Right:

```
10² = 100
```

Dono ends compare karne hain.

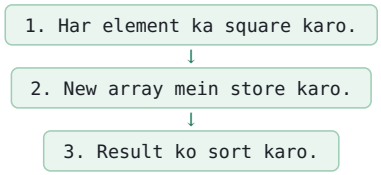
## PATTERN

Two Pointer

```
left  -> first index
right -> last index
```

Result array ke end se fill karenge.

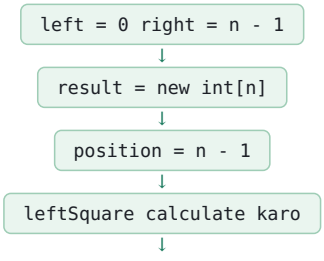
## BRUTE FORCE

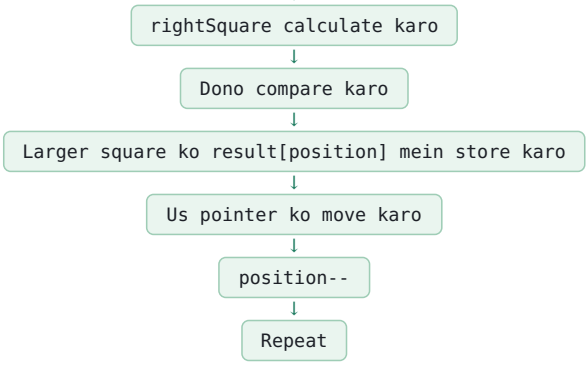



Complexity:
Time: O(n log n)
Space: O(n)

## OPTIMIZED APPROACH

Two Pointer



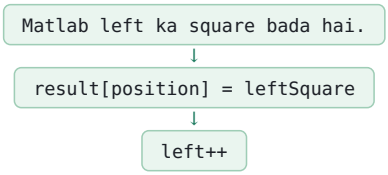



## SQUARE CALCULATION

leftSquare:
nums[left] * nums[left]
rightSquare:
nums[right] * nums[right]

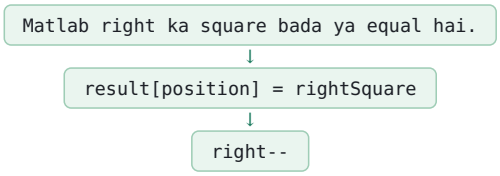
## CASE 1

leftSquare > rightSquare




## CASE 2

```
rightSquare >= leftSquare
```



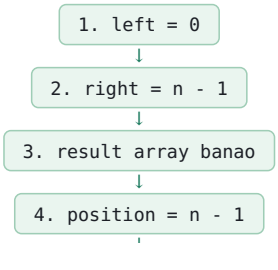

## HAR ITERATION

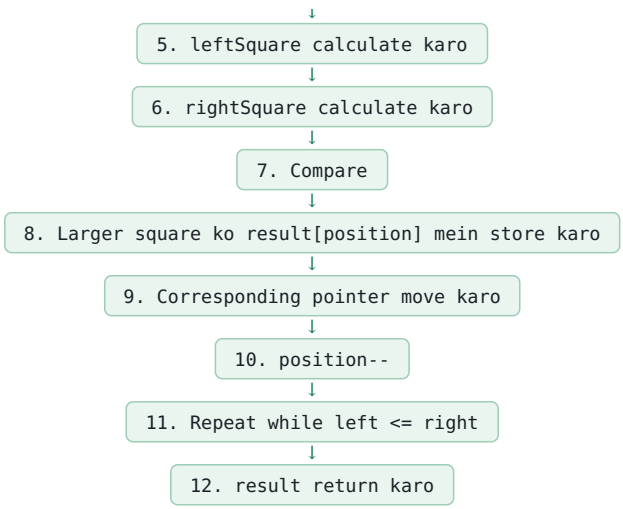
position--
WHY RESULT END SE FILL KARTE HAIN?
Har iteration mein sabse bada remaining square mil raha hai.
Isliye:

```
Largest → last position

Second largest → second-last

Third largest → third-last
```

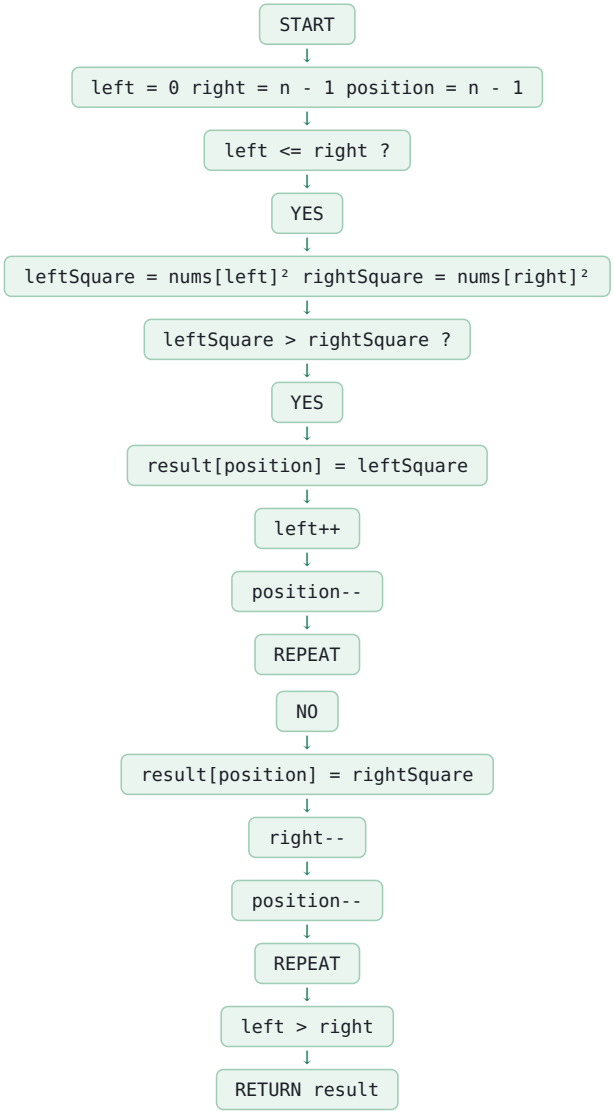
Example:
[-4, -1, 0, 3, 10]
First:
16 vs 100
100 largest hai.
result:
[_, _, _, _, 100]
Then:
16 vs 9
16 largest hai.
result:
[_, _, _, 16, 100]
Eventually:
[0, 1, 9, 16, 100]

## STEP-BY-STEP






## FLOWCHART




## DRY RUN

nums:
[-4, -1, 0, 3, 10]

## START

```
left = 0
right = 4
position = 4
```

Iteration 1:
leftSquare:

```
(-4)² = 16
```

rightSquare:

```
10² = 100
```

100 bigger.

```
result[4] = 100

right--

right = 3

position = 3
```

Iteration 2:
leftSquare:

```
(-4)² = 16
```

rightSquare:

```
3² = 9
```

16 bigger.

```
result[3] = 16

left++

left = 1

position = 2
```

Iteration 3:
leftSquare:

```
(-1)² = 1
```

rightSquare:

```
3² = 9
```

9 bigger.

```
result[2] = 9

right--

right = 2

position = 1
```

Iteration 4:
leftSquare:

```
0² = 0
```

rightSquare:

```
0² = 0
```

Equal.
Else case:

```
result[1] = 0

right--

right = 1

position = 0
```

Iteration 5:
leftSquare:

```
(-1)² = 1
```

rightSquare:

```
(-1)² = 1

result[0] = 1

right--
```

STOP.

> **NOTE**
> Dry run mein pointer movement ko carefully track karna important hai.
> Final sorted result:
> [1, 0, 9, 16, 100]
> For this particular sequence, the above dry-run ordering reveals that when equal squares occur, we must ensure the remaining smaller values are placed correctly.

## STANDARD CORRECT DRY RUN

For:

[-4, -1, 0, 3, 10]

Iteration 1:

```
16 vs 100
→ result[4] = 100
→ right--
```

Iteration 2:

```
16 vs 9
→ result[3] = 16
→ left++
```

Iteration 3:

```
1 vs 9
→ result[2] = 9
→ right--
```

Iteration 4:

```
1 vs 0
→ result[1] = 1
→ left++
```

Iteration 5:

```
0 vs 0
→ result[0] = 0
```

Final:

[0, 1, 9, 16, 100]

## COMPLETE FUNCTION

```
public static int[] sortedSquares(int[] nums) {


int left = 0;

int right = nums.length - 1;



int[] result = new int[nums.length];



int position = nums.length - 1;



while (left <= right) {



    int leftSquare = nums[left] * nums[left];

    int rightSquare = nums[right] * nums[right];



    if (leftSquare > rightSquare) {



        result[position] = leftSquare;

        left++;



    }

    else {



        result[position] = rightSquare;

        right--;

    }



    position--;

}
```

```
return result;

}
```

## COMPLEXITY

Time:

O(n)

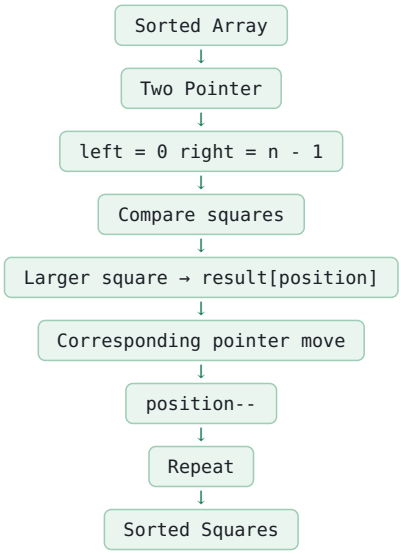
Har element maximum ek baar process hota hai.

Space:

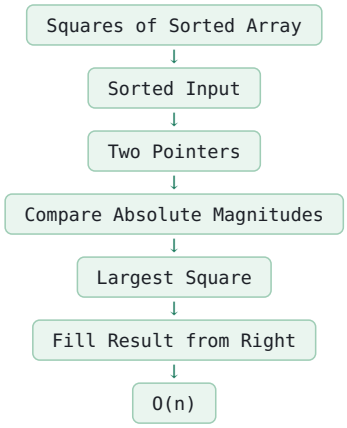
O(n)

Result array use ho raha hai.

## CORE LOGIC




## INTERVIEW PATTERN




## COMMON MISTAKES

1. Input ko directly square karke sorted maan lena.

Wrong:

[-4, -1, 0, 3]

Squares:

[16, 1, 0, 9]

Ye sorted nahi hai.

2. Result ko left se fill karna.

Largest square pehle mil raha hai.

Isliye result ko right se fill karo.

3. Negative values ka square correctly calculate na karna.

Correct:

nums[left] * nums[left]

4. `position--` bhool jaana.

Har iteration ke baad:

position--

5. Wrong loop condition.

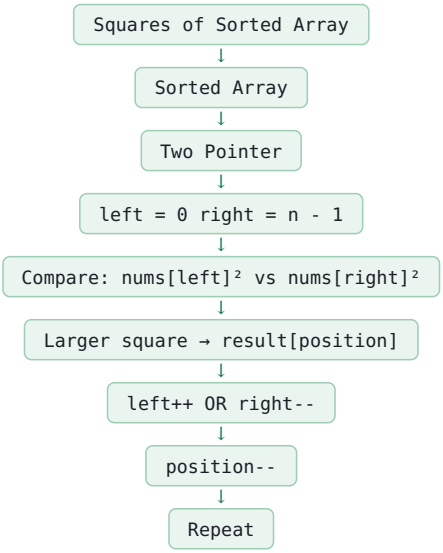
Correct:

```
while (left <= right)
```

```
WHY `<=`?
```

Jab last ek element bachta hai, left aur right same ho sakte hain.
Us element ko bhi process karna hai.

## FINAL REVISION




```
Time  = O(n)
Space = O(n)
```

# Q6 Container With Most Water

## INPUT

```
Integer array height
```

## OUTPUT

Maximum amount of water that can be stored between two lines.

## EXAMPLE

Input:
[1, 8, 6, 2, 5, 4, 8, 3, 7]
Output:
49
WHY 49?
Choose:

```
height[1] = 8
height[8] = 7
```

Width:

```
8 - 1 = 7
```

Container height:

```
min(8, 7) = 7
```

Area:

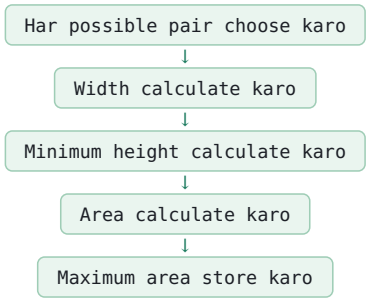
```
7 × 7 = 49
```

## FORMULA

```
Area = Width × Height
```

Width:
right - left
Height:
min(height[left], height[right])
Therefore:
area = (right - left) × min(height[left], height[right])

## BRUTE FORCE




Example:

```
i = 0
j = 1, 2, 3, ...
```

Then:

```
i = 1
j = 2, 3, 4, ...
```

Complexity:
Time: $O(n^2)$
Space: O(1)

## OPTIMIZED APPROACH

Two Pointer

```
left = 0
right = n - 1
```

Example:
[1, 8, 6, 2, 5, 4, 8, 3, 7] ↑ ↑ left right

## STEP 1

left aur right ko opposite ends par rakho.

### STEP 2

Width calculate karo:

```
width = right - left
```

### STEP 3

Container ki height:

```
h = min(height[left], height[right])
```

### STEP 4

Area:

```
area = width × h
```

### STEP 5

Maximum area update karo:

```
maxArea = Math.max(maxArea, area)
```

### STEP 6

Smaller height wala pointer move karo.

### CASE 1

height[left] < height[right]

```
→ left++
```

### CASE 2

```
height[left] >= height[right]

→ right--
```

WHY SMALLER HEIGHT POINTER MOVE KARTE HAIN?

Example:

```
left height = 3
right height = 8
```

Current height:

```
min(3, 8) = 3
```

Agar right ko move karenge, width decrease hogi.

Lekin left ki height abhi bhi 3 hai, isliye container ki maximum possible height 3 hi rahegi.

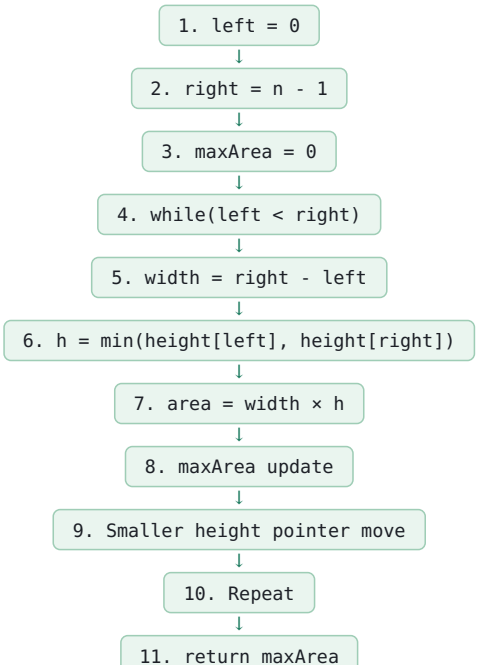
Isliye smaller height ko move karna better possibility deta hai.

Goal:

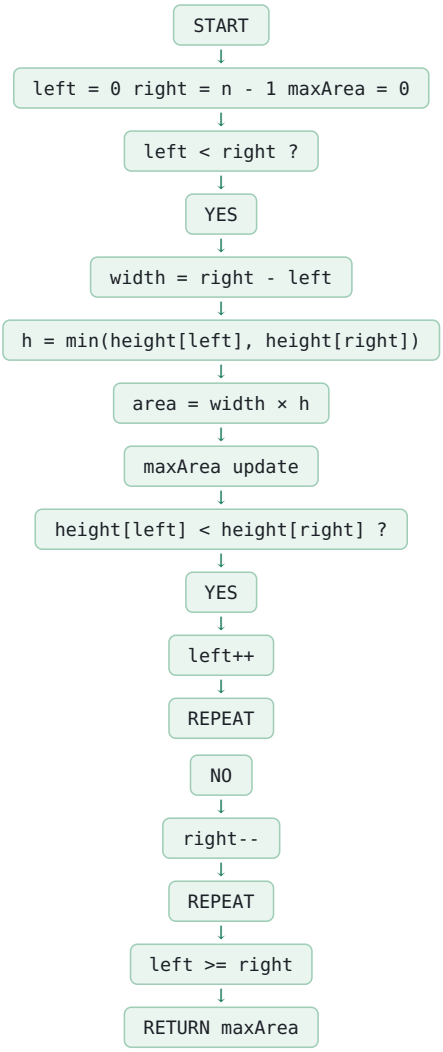
Smaller height se bigger height find karna.

> **IMPORTANT**
>
> Pointer move karne se PEHLE area calculate karna hai.
>
> Correct order:
>
> 1. Width 2. Height 3. Area 4. maxArea update 5. Pointer move

### STEP-BY-STEP



## FLOWCHART




## DRY RUN

height:

[1, 8, 6, 2, 5, 4, 8, 3, 7]

## START

```
left = 0
right = 8
```

Iteration 1:

```
height[left] = 1
height[right] = 7
```

width:

```
8 - 0 = 8
```

height:

```
min(1, 7) = 1
```

area:

```
8 × 1 = 8
```

maxArea:

8

```
Smaller height = 1

→ left++
```

Iteration 2:

```
left = 1
right = 8
```

8 vs 7

width:

```
8 - 1 = 7
```

height:

```
min(8, 7) = 7
```

area:

```
7 × 7 = 49
```

maxArea:
49

```
Smaller height = 7

→ right--
```

Continue the same process until:

```
left >= right
```

## FINAL ANSWER

49

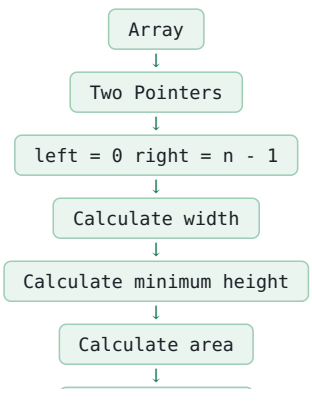
## COMPLETE FUNCTION

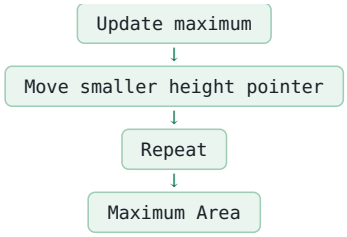
```
public static int maxArea(int[] height) {

int left = 0;
int right = height.length - 1;

int maxArea = 0;

while (left < right) {

    int h = Math.min(
        height[left],
        height[right]
    );

    int width = right - left;

    int area = width * h;

    maxArea = Math.max(
        maxArea,
        area
    );

    if (height[left] < height[right]) {

        left++;

    } else {

        right--;
    }
}

return maxArea;

}
```

## COMPLEXITY

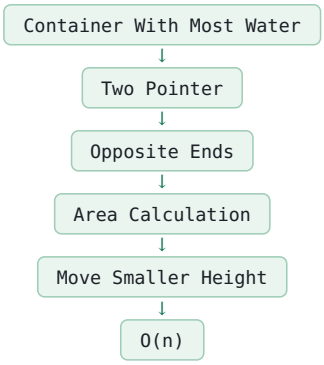
Time:
O(n)
Why?
Har iteration mein kam se kam ek pointer move hota hai.
left aur right sirf ek direction mein move karte hain.
Space:
O(1)
Koi extra array ya HashMap nahi hai.

## CORE LOGIC






## INTERVIEW PATTERN

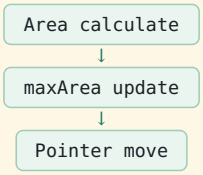



## COMMON MISTAKES

1. Height ko:
max(height[left], height[right])
karna.
Wrong.
Correct:
min(height[left], height[right])
2. Width ko:
right + left
karna.
Wrong.
Correct:
right - left
3. Area calculate karne ke baad maxArea update na karna.
Correct:

```
maxArea = Math.max(maxArea, area)
```

4. Dono pointers ko ek saath move karna.
Wrong.
Sirf smaller height wala pointer move karo.
5. Pointer move karne ke baad area calculate karna.
Wrong.
Correct order:



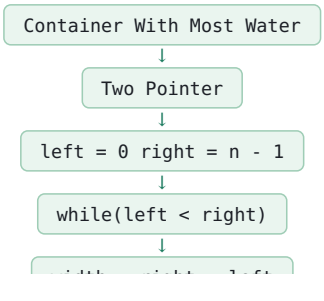

6. Indices compare karna:

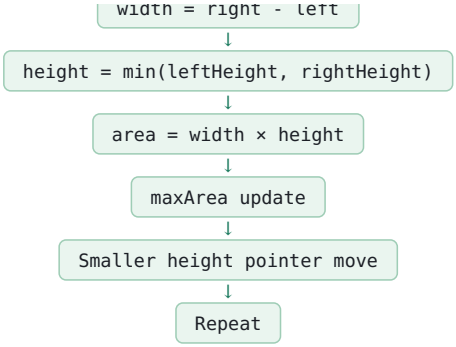
```
if (left < right)
```

Ye sirf while condition hai.
Pointer movement ke liye:

```
if (height[left] < height[right])
```

## FINAL REVISION






```
Time  = O(n)
Space = O(1)
```

# Q7 3Sum

## INPUT

```
Integer array nums
```

## OUTPUT

All unique triplets jinka sum 0 ho.

## EXAMPLE

Input:
[-1, 0, 1, 2, -1, -4]
Output:
[[-1, -1, 2], [-1, 0, 1]]
WHY?

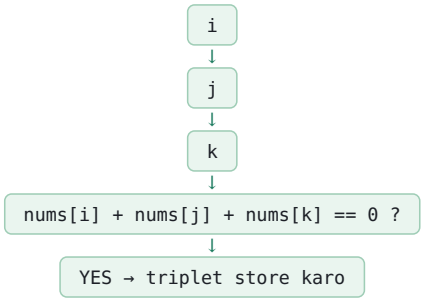
```
-1 + -1 + 2 = 0

-1 + 0 + 1 = 0
```

**IMPORTANT**

Duplicate triplets allowed nahi hain.
Example:
[-1, 0, 1]
Agar same triplet multiple times milta hai to output mein sirf ek baar hona chahiye.

## BRUTE FORCE

3 nested loops use karo.




## COMPLEXITY

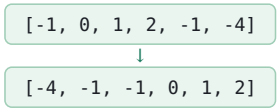
Time: $O(n^3)$
Space: O(1) extra (output space excluded)

**PROBLEM WITH BRUTE FORCE**

$O(n^3)$ bahut slow ho sakta hai.
Isliye optimize karenge.

## OPTIMIZED APPROACH

Sorting + Two Pointer
Step 1:
Array sort karo.
Example:




Step 2:
Ek element fix karo.

```
i = 0
```

Remaining array par Two Pointer lagao.

## POINTERS

```
left = i + 1

right = n - 1
```

## FORMULA

sum = nums[i] + nums[left] + nums[right]

## CASE 1

sum < 0

Sum chhota hai.
Sorted array hone ki wajah se sum increase karne ke liye:

```
left++
```

### CASE 2

sum > 0
Sum bada hai.
Sum decrease karne ke liye:

```
right--
```

### CASE 3

```
sum == 0
```

Triplet mil gaya.
Triplet store karo.
Then:

```
left++
right--
```

WHY SORTING?
Sorting ki wajah se hum Two Pointer use kar sakte hain.
Agar:
sum < 0
to left increase karne se value increase hogi.
Agar:
sum > 0
to right decrease karne se value decrease hogi.

### DUPLICATE HANDLING

3Sum mein duplicate handling bahut important hai.
DUPLICATE i:
Example:
[-1, -1, 0, 1]
Agar first -1 ke liye triplets check kar liye hain,
to second same -1 ke liye same work nahi karna.
Condition:

```
if (i > 0 && nums[i] == nums[i - 1]) {
continue;
}
```

### DUPLICATE LEFT

Triplet milne ke baad:
[-1, -1, 2, 2]
Agar left same value par rahega to duplicate triplet milega.
Isliye:

```
while (left < right &&
nums[left] == nums[left + 1]) {
left++;
}
```

### DUPLICATE RIGHT

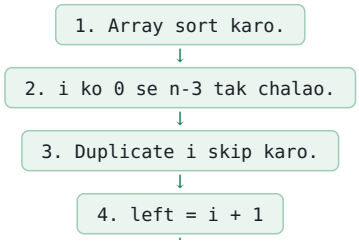
Similarly:

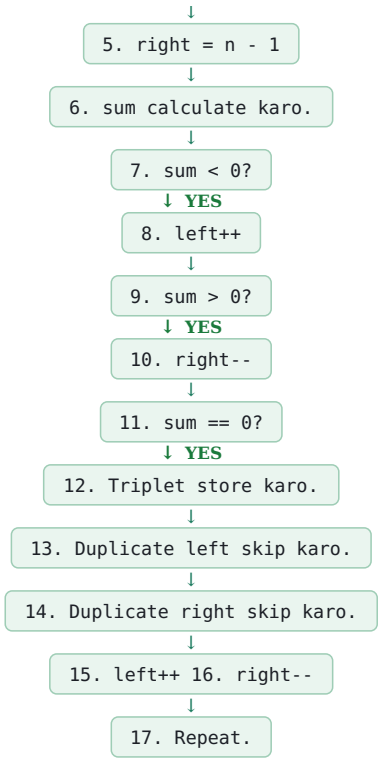
```
while (left < right &&
nums[right] == nums[right - 1]) {
right--;
}
```

After skipping duplicates:

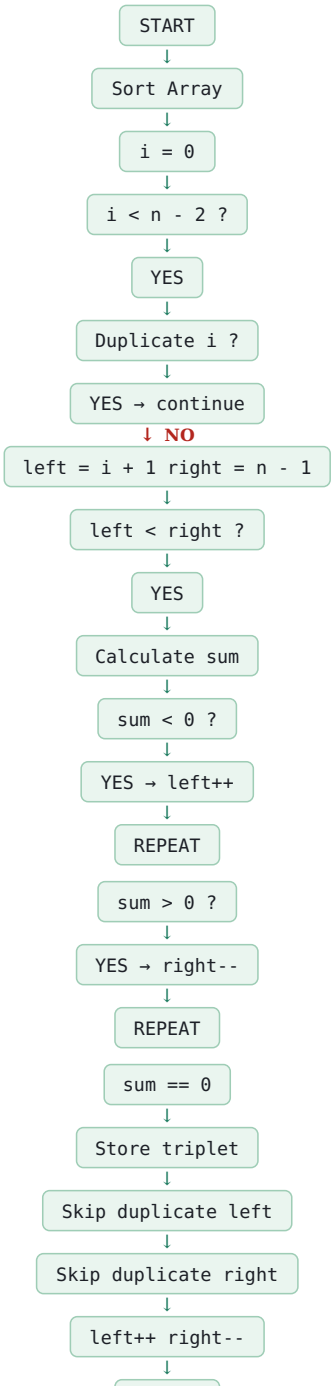
```
left++;
right--;
```

### STEP-BY-STEP




5. right = n - 1
6. sum calculate karo.
7. sum < 0?
YES
8. left++
9. sum > 0?
YES
10. right--
11. sum == 0?
YES
12. Triplet store karo.
13. Duplicate left skip karo.
14. Duplicate right skip karo.
15. left++ 16. right--
17. Repeat.

## FLOWCHART


START
Sort Array
i = 0
i < n - 2 ?
YES
Duplicate i ?
YES → continue
NO
left = i + 1 right = n - 1
left < right ?
YES
Calculate sum
sum < 0 ?
YES → left++
REPEAT
sum > 0 ?
YES → right--
REPEAT
sum == 0
Store triplet
Skip duplicate left
Skip duplicate right
left++ right--

REPEAT

## DRY RUN

Input:

[-1, 0, 1, 2, -1, -4]

## SORT

[-4, -1, -1, 0, 1, 2]

```
i = 0:

nums[i] = -4

left = 1
right = 5

-4 + -1 + 2
= -3
```

sum < 0

```
→ left++
```

Continue...

```
i = 1:

nums[i] = -1

left = 2
right = 5

-1 + -1 + 2
= 0
```

Triplet:

[-1, -1, 2]

Store.

Skip duplicates.

```
left++
right--
```

Continue:

```
-1 + 0 + 1
= 0
```

Triplet:

[-1, 0, 1]

Store.

Final:

[[-1, -1, 2], [-1, 0, 1]]

## COMPLETE FUNCTION

```
public static List<List<Integer>> threeSum(int[] nums) {

List<List<Integer>> result = new ArrayList<>();

Arrays.sort(nums);

for (int i = 0; i < nums.length - 2; i++) {

    // Duplicate first element skip
    if (i > 0 && nums[i] == nums[i - 1]) {
        continue;
    }

    int left = i + 1;
    int right = nums.length - 1;

    while (left < right) {

        int sum =
            nums[i]
            + nums[left]
            + nums[right];

        if (sum < 0) {

            left++;

        }
        else if (sum > 0) {

            right--;

        }
```

```
        else {

            result.add(
                Arrays.asList(
                    nums[i],
                    nums[left],
                    nums[right]
                )
            );

            // Skip duplicate left values
            while (left < right &&
                   nums[left] == nums[left + 1]) {
                left++;
            }

            // Skip duplicate right values
            while (left < right &&
                   nums[right] == nums[right - 1]) {
                right--;
            }

            left++;
            right--;
        }
    }
}

return result;

}
```

## COMPLEXITY

Sorting:

O(n log n)

Two Pointer for every i:

$O(n^2)$

Total:

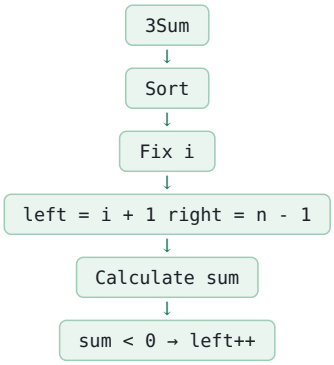
$O(n^2)$

Extra Space:

O(1)

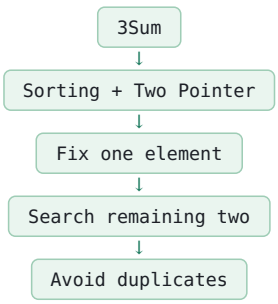
Output list ki space ko complexity mein normally separately count kiya jata hai.

## CORE LOGIC




```
sum > 0
→ right--

sum == 0
→ store triplet
→ skip duplicates
→ left++
→ right--
```

## INTERVIEW PATTERN




### COMMON MISTAKES

1. Array sort na karna.

Two Pointer ka logic sorting par depend karta hai.

2. `i` ke duplicates skip na karna.
Correct:

```
if (i > 0 && nums[i] == nums[i - 1]) {
continue;
}
```

3. `sum < 0` par right-- karna.
Wrong.
Correct:

```
sum < 0
→ left++
```

4. `sum > 0` par left++ karna.
Wrong.
Correct:

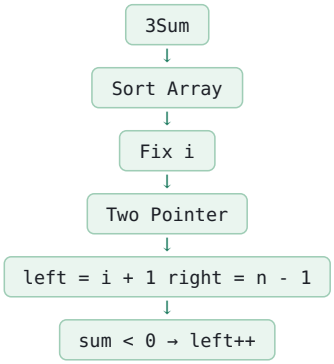
```
sum > 0
→ right--
```

5. Triplet milne ke baad duplicates skip na karna.
Duplicate triplets aa sakte hain.
6. `left++` aur `right--` triplet milne ke baad na karna.
Correct:

```
left++;
right--;
```

7. `i < nums.length` tak loop chalana.
Correct:
i < nums.length - 2
Kyuki minimum 3 elements chahiye.

## FINAL REVISION




```
sum > 0
→ right--

sum == 0
→ store
→ skip duplicates
→ left++
→ right--

Time  = O(n²)
Space = O(1) extra
```

# Q8 Sort Colors

## INPUT

```
Integer array nums
```

Array mein sirf:
0 1 2
hain.

## OUTPUT

Array ko in-place sort karna hai:

```
0 → 1 → 2
```

## EXAMPLE

Input:
[2, 0, 2, 1, 1, 0]
Output:
[0, 0, 1, 1, 2, 2]

### IMPORTANT

Built-in sorting use nahi karni.
Wrong:

```
Arrays.sort(nums);
```

### GOAL

Time: O(n)
Space: O(1)

## BRUTE FORCE

Normal sorting algorithm use karo.
Example:

```
Arrays.sort(nums)
```

Time:
O(n log n)
Lekin optimal solution:
O(n)

## PATTERN

Dutch National Flag Algorithm
3 Pointers:
low mid high

## INITIAL

```
low = 0
mid = 0
high = nums.length - 1
```

## ARRAY REGIONS

0 region | 1 region | unknown region | 2 region

```
     low        mid             high
```

More clearly:
[ 0s ][ 1s ][ unknown ][ 2s ] ↑ ↑ ↑ low mid high

## POINTER MEANING

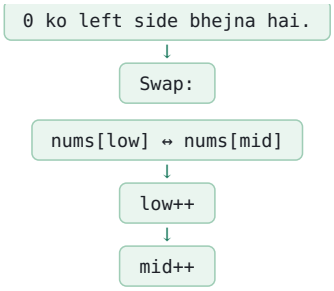
low:
Next position jahan 0 place karna hai.
mid:
Current element jo process karna hai.
high:
Next position jahan 2 place karna hai.

## MAIN LOOP

```
while (mid <= high)
```

## CASE 1

```
nums[mid] == 0
```




WHY MID++?
Swap ke baad mid position par jo element aaya hai, woh already processed/known position se aaya hai.
Isliye next element par ja sakte hain.

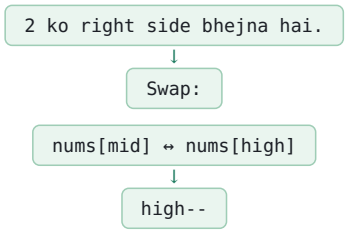
### CASE 2

```
nums[mid] == 1
```

1 already middle region mein hai.

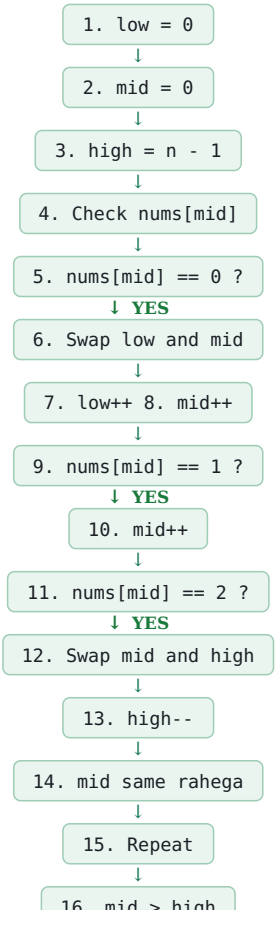

### CASE 3

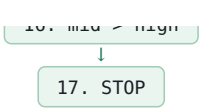
```
nums[mid] == 2
```




**IMPORTANT**
Yahan mid++ NAHI karna.
WHY?
High se jo element mid position par aaya hai woh unknown hai.
Woh 0, 1 ya 2 kuch bhi ho sakta hai.
Isliye same mid ko dobara check karna hai.

### STEP-BY-STEP

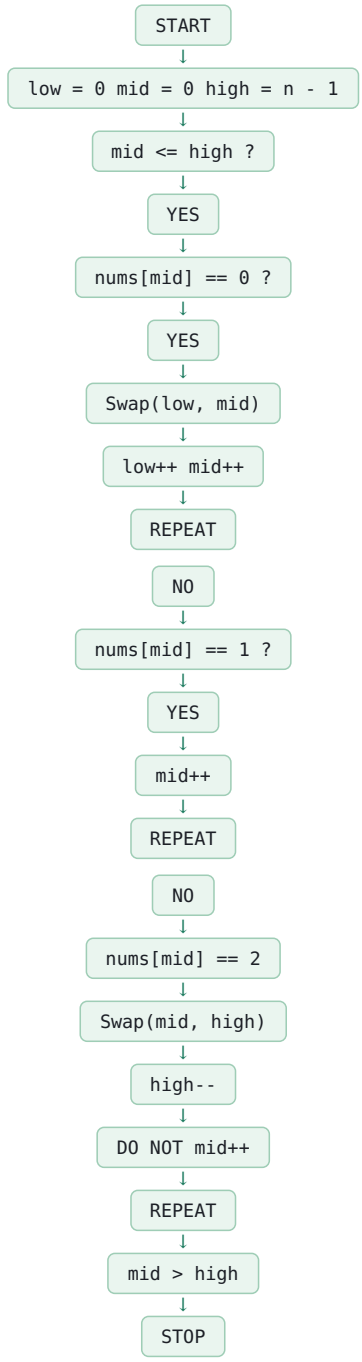





↓

17. STOP

## FLOWCHART




## DRY RUN

Input:

[2, 0, 2, 1, 1, 0]

## START

```
low = 0
mid = 0
high = 5
```

Iteration 1:

```
nums[mid] = 2
```

2 ko right bhejna hai.

Swap:

nums[0] ↔ nums[5]

Array:

[0, 0, 2, 1, 1, 2]

high--

```
high = 4

mid same = 0
```

Iteration 2:

```
nums[mid] = 0
```

Swap:
nums[low] ↔ nums[mid]
Array:
[0, 0, 2, 1, 1, 2]
low++ mid++

```
low = 1
mid = 1
```

Iteration 3:

```
nums[mid] = 0
```

Swap:
nums[1] ↔ nums[1]
Array:
[0, 0, 2, 1, 1, 2]
low++ mid++

```
low = 2
mid = 2
```

Iteration 4:

```
nums[mid] = 2
```

Swap:
nums[2] ↔ nums[4]
Array:
[0, 0, 1, 1, 2, 2]
high--

```
high = 3

mid same = 2
```

Iteration 5:

```
nums[mid] = 1
```

1 already correct region mein hai.
mid++

```
mid = 3
```

Iteration 6:

```
nums[mid] = 1
```

mid++

```
mid = 4
```

Now:
mid > high
4 > 3
STOP.

## FINAL

[0, 0, 1, 1, 2, 2]

## COMPLETE FUNCTION

```
public static void sortColors(int[] nums) {

int low = 0;
int mid = 0;
int high = nums.length - 1;

while (mid <= high) {

    if (nums[mid] == 0) {

        int temp = nums[low];
        nums[low] = nums[mid];
        nums[mid] = temp;

        low++;
        mid++;

    }
```

```
    else if (nums[mid] == 1) {

        mid++;

    }
    else {

        int temp = nums[mid];
        nums[mid] = nums[high];
        nums[high] = temp;

        high--;
    }
}

}
```

## COMPLEXITY

Time:

O(n)

Har element maximum constant number of times process hota hai.

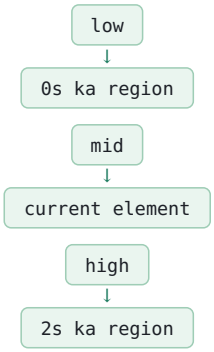
Space:

O(1)

No extra array used.

## CORE LOGIC

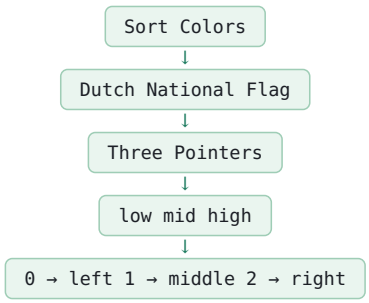
```
0 → left
1 → middle
2 → right
```




## CASE SUMMARY

```
nums[mid] == 0

→ swap(low, mid)
→ low++
→ mid++

nums[mid] == 1

→ mid++

nums[mid] == 2

→ swap(mid, high)
→ high--
→ mid same
```

## INTERVIEW PATTERN




## COMMON MISTAKES

```
1. nums[mid] == 2 ke case mein
   mid++ kar dena.
```

WRONG.

Correct:

high--

mid same.

```
2. 0 ke case mein
   low++ bhool jaana.
```

Correct:
low++ mid++

```
3. 1 ke case mein swap karna.
```

Unnecessary.
Sirf:
mid++
4. while condition:
mid < high
Wrong.
Correct:

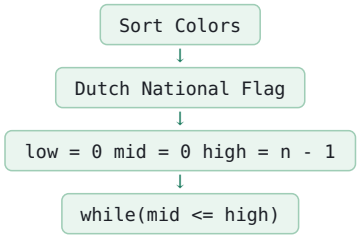
```
mid <= high

5. Arrays.sort() use karna.
```

Question ka purpose O(n) in-place sorting hai.
WHY THIS IS O(n)?
Teen pointers hain:

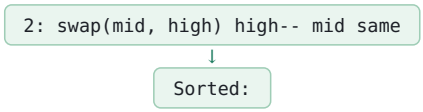
```
low → only forward
mid → only forward
high → only backward
```

Koi pointer unnecessarily backtrack nahi karta.

## FINAL REVISION




0: swap(low, mid) low++ mid++
1: mid++




[0s][1s][2s]

```
Time  = O(n)
Space = O(1)
```

# Q9 3Sum Closest

## INPUT

```
Integer array nums
Integer target
```

## OUTPUT

```
3 numbers ka aisa sum return karna hai
jo target ke sabse closest ho.
```

## EXAMPLE

Input:

```
nums = [-1, 2, 1, -4]
target = 1
```

Output:

2

WHY?

Possible sums:

```
-1 + 2 + 1 = 2
-1 + 2 + -4 = -3
-1 + 1 + -4 = -4
2 + 1 + -4 = -1
```

Target:

1

Closest sum:

2

Difference:

```
|2 - 1| = 1
```

## ANOTHER EXAMPLE

```
nums = [0, 0, 0]
target = 1
```

Output:

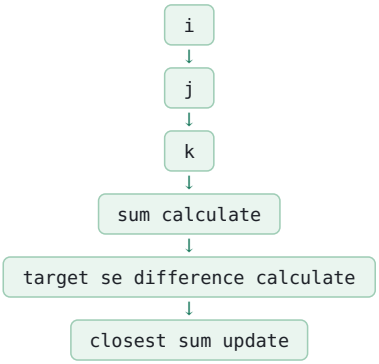
0

Because:

```
0 + 0 + 0 = 0
```

## BRUTE FORCE

3 nested loops:




## COMPLEXITY

Time:

$O(n^3)$

Space:

O(1) extra

## PROBLEM

$O(n^3)$ slow hai.

Isliye optimize karenge.

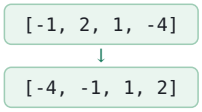
## OPTIMIZED APPROACH

Sorting + Two Pointer

## STEP 1

Array sort karo.

Example:




### STEP 2

Ek element fix karo.

i

### STEP 3

Remaining elements ke liye:

```
left = i + 1
right = n - 1
```

### STEP 4

Sum calculate karo:

sum = nums[i] + nums[left] + nums[right]

### STEP 5

Current sum aur target ka difference compare karo.

Difference:

```
Math.abs(sum - target)
```

Agar current sum ka difference closest ke difference se chhota hai:

```
closest = sum
```

### CASE 1

sum < target

Sum target se chhota hai.

Sum increase karne ke liye:

```
left++
```

### CASE 2

sum > target

Sum target se bada hai.

Sum decrease karne ke liye:

```
right--
```

### CASE 3

```
sum == target
```

Exact target mil gaya.

Isse closer koi answer possible nahi hai.

Therefore:

```
return target
```

#### IMPORTANT DIFFERENCE FROM 3SUM

3Sum:

```
sum == 0
→ triplet store
```

3Sum Closest:

```
sum == target
→ target return
```

#### CORE FORMULA

```
difference =
Math.abs(sum - target)
```

### CLOSEST UPDATE

```
if (
Math.abs(sum - target)
<
Math.abs(closest - target)
) {
```

```
closest = sum;
```

```
}
```

## INITIAL CLOSEST

closest ko first valid 3 elements ke sum se initialize karo.

Example:

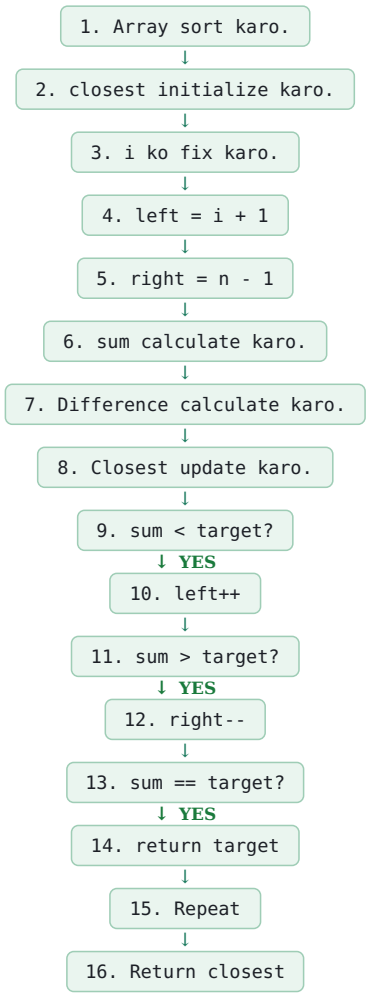
```
nums = [-4, -1, 1, 2]
```

closest:

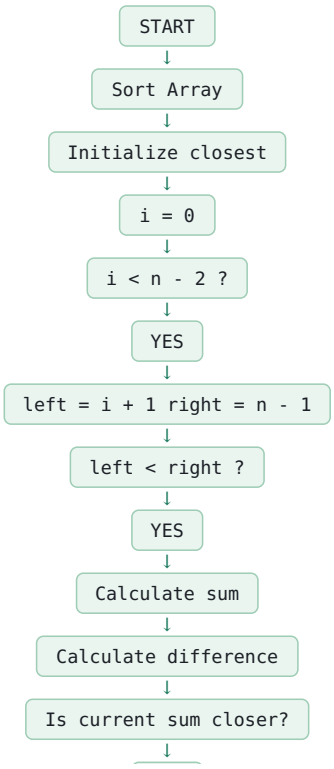
nums[0] + nums[1] + nums[2]

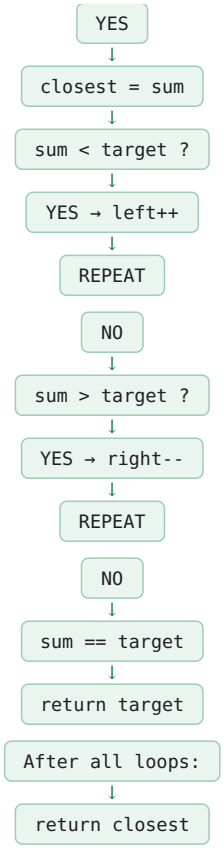
```
= -4 + -1 + 1

= -4
```

## STEP-BY-STEP




## FLOWCHART






## DRY RUN

Input:

```
nums = [-1, 2, 1, -4]
target = 1
```

## SORT

[-4, -1, 1, 2]

## INITIAL

closest:

-4 + -1 + 1

```
= -4

i = 0

left = 1
right = 3
```

Iteration 1:

sum:

-4 + -1 + 2

```
= -3
```

Difference:

|-3 - 1|

```
= 4
```

Current closest:

-4

Difference:

|-4 - 1|

```
= 5
```

4 < 5

Therefore:

```
closest = -3
```

sum < target

-3 < 1

Therefore:

```
left++
```

Iteration 2:

```
left = 2
right = 3
```

sum:
-4 + 1 + 2

```
= -1
```

Difference:
|-1 - 1|

```
= 2
```

Current closest:
-3
Difference:
4
2 < 4
Therefore:

```
closest = -1
```

sum < target
-1 < 1
Therefore:

```
left++

No more valid left/right
for i = 0.
```

Continue with next i.
Eventually:
-1 + 1 + 2

```
= 2
```

Difference:
|2 - 1|

```
= 1
```

Closest:
2
No sum gets closer.

## FINAL

2

## COMPLETE FUNCTION

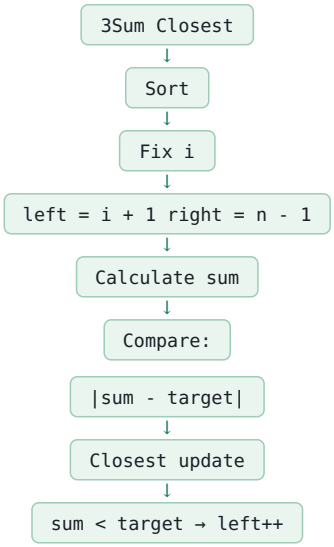
```
public static int threeSumClosest(
int[] nums,
int target) {

Arrays.sort(nums);

int closest =
    nums[0]
    + nums[1]
    + nums[2];

for (int i = 0;
     i < nums.length - 2;
     i++) {

    int left = i + 1;
    int right = nums.length - 1;

    while (left < right) {

        int sum =
            nums[i]
            + nums[left]
            + nums[right];


        // Closest sum update
        if (
            Math.abs(sum - target)
            <
            Math.abs(closest - target)
        ) {

            closest = sum;
        }
```

```
        if (sum < target) {

            left++;

        }
        else if (sum > target) {

            right--;

        }
        else {

            return target;
        }
    }
}

return closest;

}
```

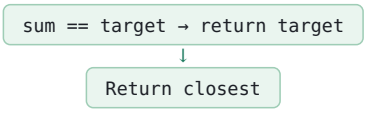
## COMPLEXITY

Sorting:
O(n log n)
Two Pointer:
$O(n^2)$
Total:
$O(n^2)$
Extra Space:
O(1)
Output ke alawa.

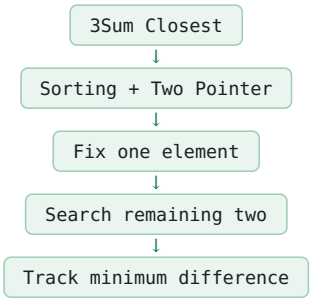
## CORE LOGIC




```
sum > target
→ right--
```




## INTERVIEW PATTERN




## COMMON MISTAKES

```
1. `closest = 0` se initialize karna.
```

Avoid.

Correct:

```
closest =
nums[0] + nums[1] + nums[2]

2. `Math.abs()` use na karna.
```

Closest ka matlab absolute difference compare karna hai.
Correct:

```
Math.abs(sum - target)
```

3. sum < target par right-- karna.
Wrong.
Correct:

```
sum < target
→ left++
```

4. sum > target par left++ karna.
Wrong.
Correct:

```
sum > target
→ right--
```

5. Exact target milne ke baad continue karna.
Unnecessary.
Correct:

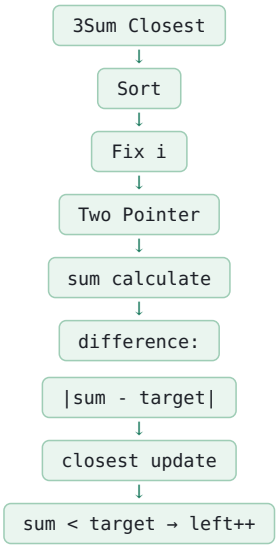
```
return target
```

6. Array sort na karna.
Two Pointer movement ke liye sorted array required hai.
7. 3 elements se kam array handle karne ki condition ignore karna.
Problem normally valid input deti hai, lekin custom code mein:

```
nums.length >= 3
```

ensure karna safe hai.

## FINAL REVISION




```
sum > target
→ right--

sum == target
→ return target

Time  = O(n²)
Space = O(1) extra
```

# Q11 Trapping Rain Water

## INPUT

```
Integer array height
```

Har element ek vertical wall/column ki height represent karta hai.

## OUTPUT

Total amount of rain water jo array ke columns ke beech trap ho sakta hai.

## EXAMPLE

Input:
[0,1,0,2,1,0,1,3,2,1,2,1]
Output:
6

## VISUAL IDEA

```
   #
```

# #
# # # # #
---
Higher walls ke beech lower positions mein water trap hota hai.

### IMPORTANT FORMULA

Kisi position par water:
water = min(leftMax, rightMax) ----------
height[i]
Example:

```
leftMax = 3
rightMax = 5
height[i] = 1
```

Water:
min(3,5) - 1

```
= 2
```

## BRUTE FORCE

Har index ke liye:
1. Left side ka maximum find karo. 2. Right side ka maximum find karo. 3. Minimum maximum height nikalo. 4. Current height subtract karo. 5. Water add karo.
Complexity:
Time: $O(n^2)$
Space: O(1)

## BETTER APPROACH

Prefix Max + Suffix Max
leftMax array banao. rightMax array banao.
Then:
water = min(leftMax[i], rightMax[i]) ----------
height[i]
Time:
O(n)
Space:
O(n)

## OPTIMIZED APPROACH

Two Pointer
Extra arrays use nahi karenge.
Use:
left right leftMax rightMax

## INITIAL

```
left = 0

right = height.length - 1

leftMax = 0

rightMax = 0

water = 0
```

**MAIN OBSERVATION**

Hum decide kar sakte hain ki left side process karni hai ya right side.

Condition:

```
if (height[left] <= height[right])
```

Left side process karo.

Otherwise:

Right side process karo.

WHY?

Agar:

```
height[left] <= height[right]
```

toh right side par at least height[left] jitni boundary available hai.

Isliye left side ke liye leftMax enough information provide karta hai.

Similarly:

height[right] < height[left]

toh right side process kar sakte hain using rightMax.

## CASE 1

```
height[left] <= height[right]
```

Left side process karo.

Check:

```
height[left] >= leftMax
```

### YES

```
Current wall new maximum hai.
```

Therefore:

```
leftMax = height[left]
```

### NO

Current wall leftMax se chhoti hai.

Water trap hoga:

```
water += leftMax - height[left]
```

Then:

```
left++
```

## CASE 2

height[left] > height[right]

Right side process karo.

Check:

```
height[right] >= rightMax
```

### YES

```
rightMax = height[right]
```

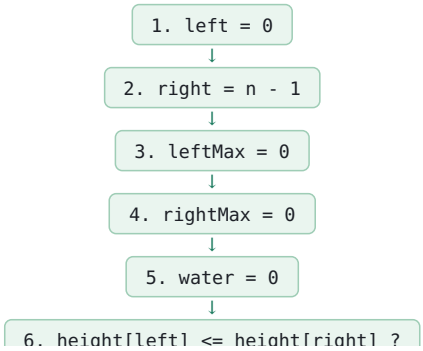
### NO

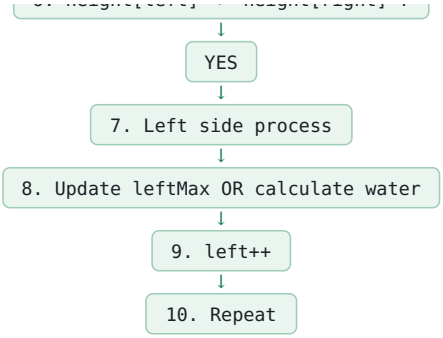
```
water += rightMax - height[right]
```

Then:

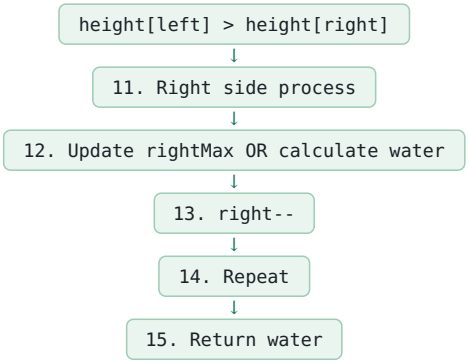
```
right--
```

## STEP-BY-STEP

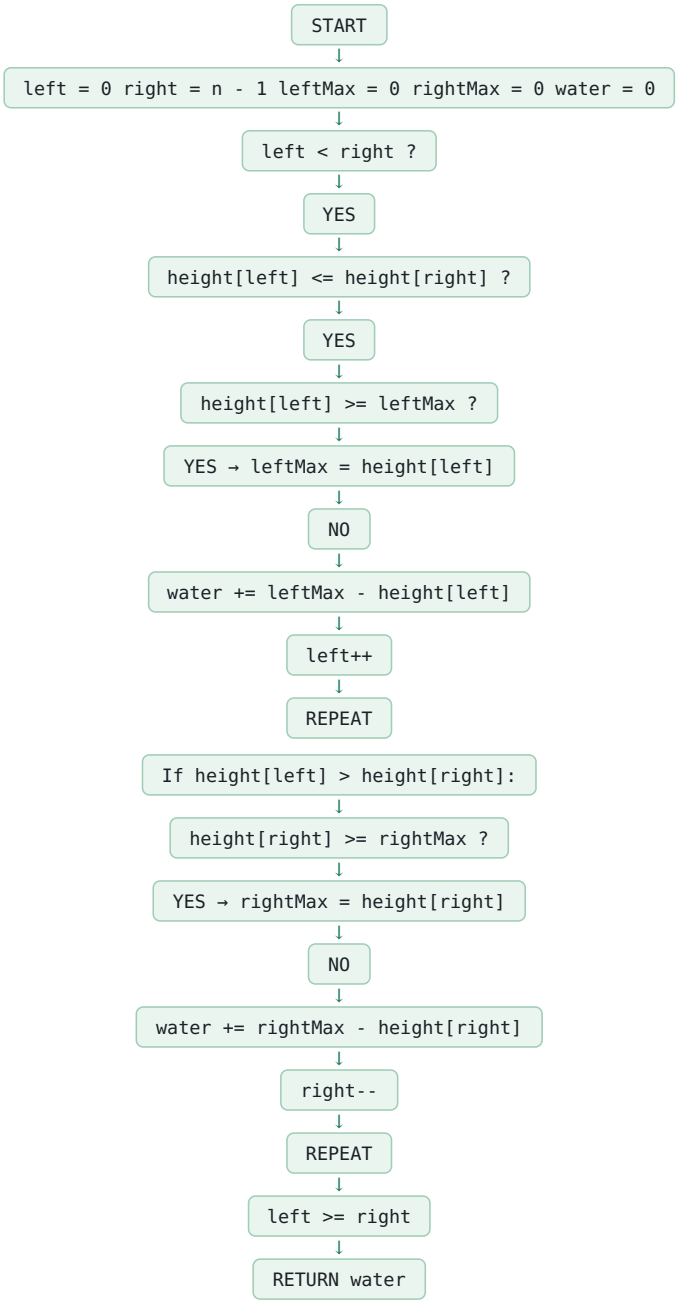





If:




## FLOWCHART




## DRY RUN

Input:

[0,1,0,2,1,0,1,3,2,1,2,1]

## START

```
left = 0
right = 11

leftMax = 0
rightMax = 0

water = 0
```

At height 0:
left side process.

```
height[left] >= leftMax

0 >= 0
```

YES

```
leftMax = 0

left++
```

At height 1:

```
height[left] = 1

1 >= leftMax
```

YES

```
leftMax = 1

left++
```

At height 0:

```
height[left] = 0

0 < leftMax = 1
```

Water:

```
1 - 0 = 1

water = 1

left++
```

At height 2:

```
2 >= leftMax

leftMax = 2

left++
```

Later smaller positions are processed using the same logic.
Water accumulated eventually:
6

## FINAL ANSWER

6

## COMPLETE FUNCTION

```
public static int trap(int[] height) {

int left = 0;
int right = height.length - 1;

int leftMax = 0;
int rightMax = 0;

int water = 0;

while (left < right) {

    if (height[left] <= height[right]) {

        if (height[left] >= leftMax) {

            leftMax = height[left];

        } else {

            water +=
                leftMax - height[left];
        }
```

```
        left++;

    } else {

        if (height[right] >= rightMax) {

            rightMax = height[right];

        } else {

            water +=
                rightMax - height[right];
        }

        right--;
    }
}

return water;

}
```

## COMPLEXITY

Time:

O(n)

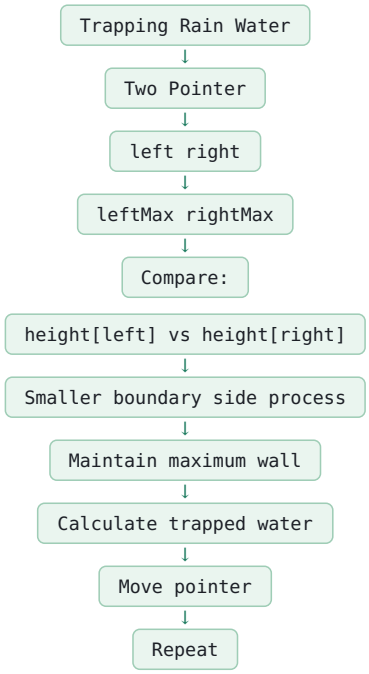
Each pointer moves only in one direction.

Space:

O(1)

No prefix/suffix arrays or extra data structures.

## CORE LOGIC




## IMPORTANT FORMULA

Left side:

if height[left] < leftMax

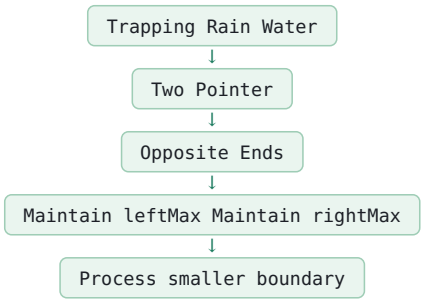
```
water +=
leftMax - height[left]
```

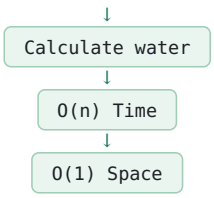
Right side:

if height[right] < rightMax

```
water +=
rightMax - height[right]
```

## INTERVIEW PATTERN






WHY SMALLER SIDE?
Water level is determined by:
min(left boundary, right boundary)
If:

```
height[left] <= height[right]
```

then left boundary is the limiting side.
We can safely calculate the left side using leftMax.
If:
height[right] < height[left]
then right boundary is limiting.
We calculate using rightMax.

**COMMON MISTAKES**

1. Current height ko directly water mein add karna.
Wrong.
Correct:
leftMax - height[left]
or:
rightMax - height[right]
2. `max()` use karna.
Wrong:
max(leftMax, rightMax)
Correct concept:
min(leftMax, rightMax) because water lower boundary par depend karta hai.
3. leftMax update bhool jaana.
Correct:

```
if (height[left] >= leftMax)
```

```
leftMax = height[left];
```

4. rightMax update bhool jaana.
Correct:

```
if (height[right] >= rightMax)
```

```
rightMax = height[right];
```

5. Smaller side ke instead random pointer move karna.
Correct:

```
height[left] <= height[right]
→ left++

Otherwise:
→ right--
```

6. Extra arrays use karna jab O(1) solution expected ho.
Prefix/suffix solution valid hai, but Two Pointer optimal hai.

## EDGE CASES

Empty array:

```
[]

→ 0
```

One element:
[5]

```
→ 0
```

Two elements:
[3,5]

```
→ 0
```

Increasing:

[1,2,3,4,5]

```
→ 0
```

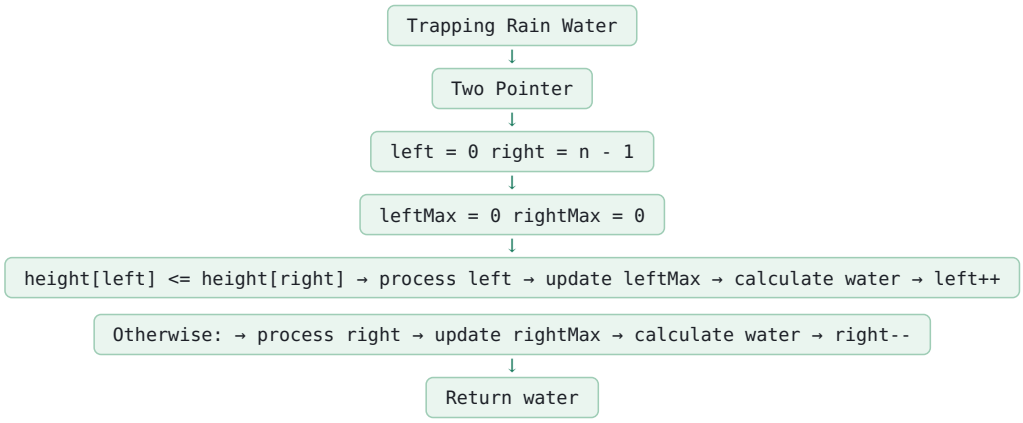
Decreasing:
[5,4,3,2,1]

```
→ 0
```

No walls:
[0,0,0]

```
→ 0
```

## FINAL REVISION




```
Time  = O(n)
Space = O(1)
```